**SKRIPSI**

# PERBANDINGAN PROSEDUR IMPERATIF TEROTOMASI BERBASIS `KUBECTL` DAN GITOPS DENGAN ARGOCD UNTUK MENGHADAPI PENYIMPANGAN KONFIGURASI PADA KLASTER KUBERNETES

## *A COMPARISON OF AN AUTOMATED `KUBECTL`-BASED IMPERATIVE PROCEDURE AND ARGOCD GITOPS FOR ADDRESSING CONFIGURATION DRIFT IN KUBERNETES CLUSTERS*




MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

**PROGRAM STUDI ILMU KOMPUTER**
**DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA**
**FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM**
**UNIVERSITAS GADJAH MADA**
**YOGYAKARTA**

**2026**

**SKRIPSI**

# PERBANDINGAN PROSEDUR IMPERATIF TEROTOMASI BERBASIS `KUBECTL` DAN GITOPS DENGAN ARGOCD UNTUK MENGHADAPI PENYIMPANGAN KONFIGURASI PADA KLASTER KUBERNETES

## *A COMPARISON OF AN AUTOMATED `KUBECTL`-BASED IMPERATIVE PROCEDURE AND ARGOCD GITOPS FOR ADDRESSING CONFIGURATION DRIFT IN KUBERNETES CLUSTERS*

Diajukan untuk memenuhi salah satu syarat memperoleh derajat
S.Kom.




MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

**PROGRAM STUDI ILMU KOMPUTER**
**DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA**
**FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM**
**UNIVERSITAS GADJAH MADA**
**YOGYAKARTA**

**2026**

# HALAMAN PENGESAHAN

## SKRIPSI

## PERBANDINGAN PROSEDUR IMPERATIF TEROTOMASI BERBASIS `KUBECTL` DAN GITOPS DENGAN ARGOCD UNTUK MENGHADAPI PENYIMPANGAN KONFIGURASI PADA KLASTER KUBERNETES

Telah dipersiapkan dan disusun oleh

MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

Telah dipertahankan di depan Tim Penguji
pada tanggal 17 Juni 2026

Susunan Tim Penguji

| | |
|---|---|
| Guntur Budi Herwanto, S.Kom., M.Cs.<br>Pembimbing | Dr. techn. Kabul Kurniawan, S.Kom., M.Cs.<br>Ketua Penguji |
| | Aina Musdholifah, S.Kom., M.Kom., Ph.D.<br>Anggota Penguji |

# PERNYATAAN

Dengan ini saya menyatakan bahwa dalam Skripsi ini tidak terdapat karya yang pernah diajukan untuk memperoleh gelar kesarjanaan di suatu Perguruan Tinggi, dan sepanjang pengetahuan saya juga tidak terdapat karya atau pendapat yang ditulis atau diterbitkan oleh orang lain, kecuali yang secara tertulis diacu dalam naskah ini dan disebutkan dalam daftar pustaka.

Yogyakarta, 17 Juni 2026

Muhammad Argya Vityasy

# PRAKATA

Isi Prakata di sini...

# DAFTAR ISI

**Halaman Judul** **ii**

**Halaman Pengesahan** **iii**

**Halaman Pernyataan** **iv**

**Halaman Motto** **v**

**Halaman Persembahan** **vi**

**PRAKATA** **vii**

**DAFTAR TABEL** **x**

**DAFTAR GAMBAR** **xi**

**DAFTAR LAMBANG DAN SINGKATAN** **xv**

**INTISARI** **xvi**

**ABSTRACT** **xvii**

**I PENDAHULUAN** **1**

1.1 Latar Belakang . . . . . 1
1.2 Rumusan Masalah . . . . . 2
1.3 Batasan Masalah . . . . . 3
1.4 Tujuan Penelitian . . . . . 4
1.5 Manfaat Penelitian . . . . . 4
1.6 Sistematika Penulisan . . . . . 4

**II TINJAUAN PUSTAKA** **6**

2.1 Penyimpangan Konfigurasi (*Configuration Drift*) . . . . . 6
2.1.1 Faktor yang Menimbulkan Penyimpangan Konfigurasi . . . . 6
2.2 Terminologi Dasar . . . . . 7
2.2.1 Kubernetes dan Orkestrasi Kontainer . . . . . 7
2.2.2 Git dan *Version Control* . . . . . 7

2.2.3 *Continuous Integration* dan *Continuous Deployment* . . . . . 8
2.2.4 *Infrastructure-as-Code* (IaC) . . . . . 8
2.2.5 Paradigma Deklaratif vs Imperatif . . . . . 8
2.3 Manajemen Konfigurasi Infrastruktur . . . . . 8
2.4 *Continuous Deployment* dan Paradigma GitOps . . . . . 9
2.4.1 Performa *Deployment* dan *Anti-Pattern* . . . . . 9
2.5 Dasar Teori: *Reconciliation* dan *Convergence* pada Sistem Deklaratif . 10
2.5.1 *Convergence* Menuju *Desired State* . . . . . 10
2.5.2 *Audit Trail* sebagai *Social Control Mechanism* . . . . . 10
2.6 *Runbook* sebagai Standarisasi Prosedur Pemulihan . . . . . 11
2.7 Paradigma *Deployment* Alternatif . . . . . 11
2.8 Evaluasi Empiris GitOps dan ArgoCD . . . . . 11
2.9 Metode Eksperimental Terkontrol pada Riset Sistem . . . . . 12
2.10 Klasifikasi Beban Uji (*Workload*) . . . . . 13
2.11 Ruang Kontribusi Penelitian . . . . . 14

**III METODE PENELITIAN** **15**
3.1 Deskripsi Umum dan Status Protokol . . . . . 15
3.2 Rancangan Perbandingan . . . . . 16
3.3 Alur Penelitian . . . . . 17
3.4 Beban Uji dan Lingkungan . . . . . 19
3.4.1 Gerbang Kesiapan . . . . . 19
3.5 Intervensi dan Skenario . . . . . 20
3.6 Definisi Variabel dan Metrik . . . . . 22
3.6.1 R1: Kesetaraan Konfigurasi dan Kesehatan . . . . . 22
3.6.2 R2: Deteksi, Ketersediaan Bukti, dan Pemulihan . . . . . 23
3.6.3 R3: Batas Pengukuran . . . . . 24
3.7 Protokol Pengulangan dan Penanganan Kegagalan . . . . . 24
3.8 Pengulangan dan Analisis Data . . . . . 26
3.9 Ancaman terhadap Validitas dan Keterbatasan Rancangan . . . . . 27
3.9.1 Validitas Konstruk . . . . . 27
3.9.2 Validitas Internal . . . . . 27
3.9.3 Validitas Eksternal . . . . . 28
3.9.4 Validitas Kesimpulan . . . . . 28

**IV HASIL DAN ANALISIS** **29**
4.1 Status Data dan Provenans . . . . . 29
4.2 Format Pelaporan E1: Kesetaraan dan Kesehatan (R1) . . . . . 30
4.3 Format Pelaporan E2: Waktu (R2) . . . . . 31
4.3.1 Efikasi pemulihan drift terkelola (B–E) . . . . . 31
4.4 R3: Tidak Diukur . . . . . 32
4.5 Analisis yang Diterapkan . . . . . 33
4.6 Batas Interpretasi . . . . . 33

**V KESIMPULAN DAN SARAN** **35**
5.1 Kesimpulan . . . . . 35
5.1.1 Jawaban RQ1: kesetaraan konfigurasi . . . . . 35
5.1.2 Jawaban RQ2: waktu deteksi, bukti, dan pemulihan . . . . . 36
5.1.3 R3 dan batas kesimpulan . . . . . 36
5.2 Saran . . . . . 37
5.2.1 Bagi praktisi . . . . . 37
5.2.2 Bagi peneliti . . . . . 37
5.3 Keterbatasan Penelitian . . . . . 38
5.4 Pekerjaan Lanjutan . . . . . 39
5.4.1 Replikasi dan validitas eksternal . . . . . 39
5.4.2 Pengukuran yang dapat diperluas dengan sumber daya tambahan . . . . . 39
5.4.3 Mengurangi batasan protokol . . . . . 40
5.4.4 Agenda yang diprioritaskan . . . . . 41

**A LAMPIRAN A** **46**
1.1 Identitas Apparatus yang Dibekukan . . . . . 46
1.2 Perintah Eksekusi . . . . . 46
1.3 Skema Data per Run . . . . . 47
1.4 Perlindungan Informasi . . . . . 49
1.5 Pemetaan Keterbatasan dan Penanganannya . . . . . 49
1.6 Cara Membaca Hasil . . . . . 50
1.7 Status Lampiran . . . . . 51

# DAFTAR TABEL

3.1 Skenario intervensi yang direncanakan . . . . . 21

4.1 Identitas apparatus yang dibekukan . . . . . 29
4.2 Median skor kesetaraan bidang terpilih (R1), format dasar / pascainjeksi / pascapemulihan . . . . . 30
4.3 Jumlah observasi per luaran waktu dan status pelaporan . . . . . 31
4.4 Perbandingan efikasi B–E: nilai $p$ terkoreksi Holm dan ukuran efek . . 32

5.1 Prioritas pekerjaan lanjutan . . . . . 41

1.1 Identitas apparatus pada pengumpulan formal . . . . . 46
1.2 Bidang utama pada berkas baris per run . . . . . 48
1.3 Keterbatasan, penanganan, dan sisa risiko . . . . . 50

# DAFTAR GAMBAR

3.1 Rancangan dua mode pengelolaan berurutan pada satu stack Invenio-RDM; mode imperatif adalah penjedaan ArgoCD, bukan *deployment* terpisah. . . . . 17
3.2 Bagan alir protokol dengan pembekuan sumber sebelum blok pengukuran dan tanpa surrogat pada desain formal. . . . . 18

# DAFTAR LAMBANG DAN SINGKATAN

*Control loop* Siklus berulang "amati–bandingkan–perbaiki" yang menjadi dasar kerja pengendali Kubernetes

*Deployment* Objek Kubernetes yang mengelola replika Pod dan pembaruan bertahap

*Drift* Penyimpangan keadaan nyata klaster dari keadaan yang dideklarasikan

*Imperative* Pendekatan yang menjalankan perintah eksplisit untuk mengubah keadaan, bukan menyatakan keadaan akhir

*Manifest* Berkas YAML yang mendeklarasikan satu atau beberapa objek Kubernetes

*Namespace* Pemisahan logis sumber daya di dalam satu klaster Kubernetes

*Pod* Unit terkecil yang dijalankan Kubernetes, berisi satu atau beberapa kontainer

*Polling* Pemeriksaan berkala pada selang waktu tetap, berbeda dari pengamatan berkelanjutan

*Prune* Penghapusan otomatis objek yang tidak lagi ada dalam deklarasi

*Reconciliation* Proses membandingkan keadaan nyata dengan deklarasi lalu membawanya kembali selaras

*SealedSecret* Objek Kubernetes yang menyimpan rahasia terenkripsi sehingga aman disimpan di Git

*Self-heal* Pemulihan otomatis oleh agen rekonsiliasi ketika keadaan nyata menyimpang

*Single source of truth* Satu lokasi tunggal yang dianggap otoritatif untuk suatu konfigurasi

*Stack* Kumpulan layanan yang dijalankan bersama untuk membentuk satu sistem aplikasi

*State* (keadaan) Kondisi nyata objek di klaster pada suatu waktu

*Ukuran efek* Besaran perbedaan antar kelompok, dilaporkan bersama nilai $p$ agar signifikansi tidak disalahartikan sebagai kepraktisan

*Wilcoxon signed-rank* Uji nonparametrik berpasangan untuk membandingkan dua kondisi pada subjek yang sama

ArgoCD Alat GitOps *pull-based* yang membandingkan keadaan klaster dengan deklarasi di Git lalu melakukan rekonsiliasi otomatis

Cliff's $\delta$ Ukuran efek nonparametrik berkisar $-1$ sampai $1$; nilai $1$ berarti seluruh pengamatan satu kelompok lebih besar daripada kelompok lain

ConfigMap Objek Kubernetes yang menyimpan konfigurasi non-rahasia sebagai pasangan kunci–nilai

Declarative Pendekatan yang menyatakan keadaan akhir yang diinginkan, bukan langkah-langkah untuk mencapainya

GitOps Paradigma pengelolaan infrastruktur yang menjadikan repositori Git sebagai satu-satunya sumber kebenaran, dengan agen otomatis yang menyelaraskan klaster

Holm Metode koreksi nilai $p$ untuk banyak pengujian sekaligus agar peluang positif palsu tetap terkendali

HPA *Horizontal Pod Autoscaler*: pengendali Kubernetes yang menambah atau mengurangi jumlah replika berdasarkan beban

IaC *Infrastructure-as-Code*: praktik mendefinisikan infrastruktur melalui berkas konfigurasi yang diberi versi

Kaplan–Meier Metode estimasi kurva kelangsungan hidup yang memperhitungkan pengamatan tersensor

R1 Metrik kesetaraan konfigurasi pada himpunan bidang yang ditetapkan, dilaporkan terpisah dari kesehatan layanan

R2 Metrik waktu: deteksi, ketersediaan bukti penyimpangan, dan pemulihan konfigurasi penuh

R3 Bukti keterlacakan; tidak diukur dalam studi ini karena audit API server tidak tersedia (lihat Bab V)

RMST *Restricted Mean Survival Time*: rata-rata waktu bertahan pada horizon tertentu, tahan terhadap penyensoran

Tersensor (kanan) Pengamatan yang berakhir sebelum peristiwa terjadi; diketahui nilainya melebihi batas pengamatan, tetapi nilainya tidak diketahui

# INTISARI

## Perbandingan Prosedur Imperatif Terotomasi Berbasis `kubectl` dan GitOps dengan ArgoCD untuk Menghadapi Penyimpangan Konfigurasi pada Klaster Kubernetes

Oleh

Muhammad Argya Vityasy
23/522547/PA/22475

Skripsi ini membandingkan dua mode pengelolaan konfigurasi pada satu stack InvenioRDM produksi-like: mode imperatif terotomasi berbasis `kubectl` dan mode GitOps berbasis ArgoCD. Pada mode imperatif, *auto-sync*/*self-heal* ArgoCD dijeda, pemeriksa dan *runbook* otomatis menjalankan koreksi `kubectl`, lalu ArgoCD dipulihkan; ini bukan *deployment* terpisah dan bukan studi waktu reaksi manusia. Kedua mode berjalan berurutan pada stack yang sama, dengan tujuh skenario *drift* dan lima belas pengulangan per skenario per mode, sehingga total 210 sel. Apparatus dibekukan sebelum pengumpulan pada revisi sumber `81ddd25`, SHA-256 render `a5efd711`, dan *image digest* tetap.

Dari 210 sel, 207 baris dianalisis dan empat dikecualikan sebagai kegagalan instrumentasi serta lingkungan yang dilaporkan terbuka. Pada RQ1, kedua mode mengembalikan skor kesetaraan bidang terpilih ke keadaan yang dideklarasikan pada skenario A–F; skenario G merupakan satu-satunya asimetri karena mode GitOps tidak mengembalikan anotasi di luar himpunan bidang yang dikelolanya. Pada RQ2, seluruh dua belas perbandingan B–E menolak hipotesis nol setelah koreksi Holm, tetapi selisih waktu deteksi sekitar empat detik bersesuaian dengan interval *polling* pemeriksa imperatif, sehingga perbedaan tersebut terutama mencerminkan mekanisme pengamatan. Ukuran efek pemulihan jauh lebih kecil ($\delta = 0{,}48$ untuk B dan $0{,}75$ untuk C) daripada deteksi, sehingga keunggulan waktu terkonsentrasi di awal siklus rekonsiliasi dan menyusut di akhir.

R3 tidak diukur karena audit API server tidak tersedia. Penelitian ini merupakan studi kasus eksploratif pada satu klaster dengan mode berurutan, sehingga tidak menyimpulkan keunggulan umum salah satu mode, tidak menyimpulkan kelayakan adopsi, dan tidak membuktikan kebenaran semantik konfigurasi yang dideklarasikan.

**Kata Kunci:** penyimpangan konfigurasi, Kubernetes, GitOps, otomasi imperatif, studi kasus komparatif terkontrol

# ABSTRACT

## A Comparison of an Automated `kubectl`-Based Imperative Procedure and ArgoCD GitOps for Addressing Configuration Drift in Kubernetes Clusters

By

Muhammad Argya Vityasy
23/522547/PA/22475

This thesis compares two configuration-management modes on one production-like InvenioRDM stack: an automated imperative mode based on `kubectl` and a GitOps mode based on ArgoCD. In the imperative mode, ArgoCD auto-sync/self-heal is paused, an automated checker and runbook perform `kubectl` corrections, and ArgoCD is then restored; this is neither a separate deployment nor a human-reaction-time study. Both modes run sequentially on the same stack across seven drift scenarios with fifteen repetitions per scenario per mode, totalling 210 cells. The apparatus was frozen before collection at source revision `81ddd25`, render SHA-256 `a5efd711`, and a fixed image digest.

Of the 210 cells, 207 rows were analysed and four were excluded as openly reported instrumentation and environmental failures. For RQ1, both modes returned the selected-field equality score to the declared state for scenarios A–F; scenario G is the sole asymmetry because the GitOps mode did not revert an annotation outside its managed field set. For RQ2, all twelve B–E comparisons rejected the null after Holm correction, but the roughly four-second detection gap matches the imperative checker polling interval, so that difference mainly reflects the observation mechanism. Recovery effect sizes are materially smaller ($\delta = 0.48$ for B and $0.75$ for C) than detection, so the timing advantage is concentrated at the start of the reconciliation cycle and shrinks at the end.

R3 was not measured because Kubernetes API-server audit was unavailable. This is an exploratory single-case study on one cluster with sequential arms; it therefore claims neither general superiority of either mode, nor adoption suitability, nor semantic correctness of the declared configuration.

**Keywords:** configuration drift, Kubernetes, GitOps, automated imperative management, controlled single-case comparative study

# BAB I

# PENDAHULUAN

## 1.1 Latar Belakang

Kubernetes mengelola sumber daya aplikasi melalui keadaan yang dideklarasikan dan rekonsiliasi oleh komponen pengendali. Dalam operasi nyata, keadaan aktual dapat berbeda dari deklarasi yang menjadi acuan. Perbedaan ini disebut penyimpangan konfigurasi (*configuration drift*). Penyimpangan perlu dibedakan dari cacat statis: suatu manifest dapat mengandung konfigurasi yang keliru tetapi tetap sama dengan keadaan aktual, sedangkan drift merujuk pada ketidakcocokan antara acuan dan keadaan yang berjalan. Dengan demikian, pengukuran kesetaraan konfigurasi tidak dengan sendirinya menilai apakah nilai konfigurasi tersebut baik atau aman (Zhang et al., 2026; Rahman et al., 2023).

Pengelolaan deklaratif dapat menggunakan pengendali GitOps untuk membandingkan sumber daya klaster dengan deklarasi di repositori dan mencoba merekonsiliasikannya. Pendekatan imperatif menggunakan perintah untuk mengubah keadaan klaster; perilaku setelah perubahan bergantung pada otomasi dan prosedur yang dipasang. Karena itu, perbandingan empiris perlu menetapkan dengan jelas mekanisme yang dibandingkan. Penelitian ini tidak mengukur kecepatan atau keahlian manusia. Mode pembanding imperatif menggunakan prosedur otomatis berbasis `kubectl` yang tetap dan terstandarisasi, sedangkan mode GitOps menggunakan ArgoCD dengan kebijakan rekonsiliasi yang diverifikasi sebelum eksperimen. Kedua mode dijalankan berurutan pada satu stack InvenioRDM produksi-like; mode imperatif adalah penjedaan ArgoCD, bukan *deployment* terpisah.

Sejumlah penelitian telah mengevaluasi GitOps maupun membandingkannya dengan pendekatan lain. Shrestha dan Ali membahas waktu induksi, penemuan, dan pemulihan drift dalam perbandingan GitOps dengan Ansible (Shrestha and Ali, 2024). Kagganti Nataraja mendeskripsikan perbandingan terkontrol pada Minikube satu node dengan konfigurasi keamanan yang berbeda (Nataraja, 2025). Matubber juga memasukkan pembanding lama yang semi-terotomasi dalam evaluasi infrastruktur Kubernetes (Matubber, 2025). Studi-studi tersebut menunjukkan bahwa evaluasi komparatif telah dilakukan; penelitian ini tidak mengklaim sebagai perbandingan pertama atau menganggap bahwa GitOps pasti lebih unggul. Rincian lingkup dan batas

interpretasi studi tersebut dibahas pada Bab II.

Penelitian ini merancang pengukuran per skenario pada beban uji multi-service InvenioRDM. Pengukuran membedakan kesetaraan bidang konfigurasi dari kesehatan layanan, memisahkan waktu deteksi dari ketersediaan bukti yang dapat diverifikasi, dan mengukur pemulihan terhadap kesetaraan inventaris serta bidang yang telah ditetapkan. Skenario A diperlakukan sebagai kontrol HPA, F sebagai batas visibilitas sumber daya di luar cakupan aplikasi, dan G sebagai kasus deskriptif yang bukan *self-heal* ArgoCD; ketiganya tidak dipool ke estimasi efikasi B–E. Keterlacakan (R3) tidak diukur karena audit API server Kubernetes tidak tersedia; keterbatasan ini dibahas pada Bab III dan Bab V.

Rancangan ini dimaksudkan untuk memperjelas perbedaan operasional antara dua prosedur yang ditetapkan pada konteks eksperimen tertentu. Karena kedua mode memakai satu stack secara berurutan, perlakuan tidak terisolasi dari beban latar dan dependensi bersama. Rekomendasi praktis maupun kesimpulan tentang arah perbedaan hanya dapat dibuat setelah apparatus divalidasi, protokol ditinjau, dan data formal dikumpulkan. Pada draf ini, hasil eksperimen belum tersedia.

## 1.2 Rumusan Masalah

Penelitian ini diarahkan oleh pertanyaan berikut.

1. **RQ1:** Bagaimana kesetaraan bidang konfigurasi yang ditetapkan dan kesehatan layanan, yang dilaporkan secara terpisah, berubah setelah intervensi dan prosedur koreksi pada mode imperatif terotomasi berbasis `kubectl` dibandingkan dengan mode GitOps menggunakan ArgoCD?

2. **RQ2:** Bagaimana distribusi waktu deteksi, waktu hingga tersedia bukti penyimpangan yang dapat diverifikasi, dan waktu pemulihan konfigurasi penuh berbeda antara kedua mode untuk setiap skenario yang dianalisis?

RQ1 dan RQ2 dijawab per skenario. HPA normal tetap aktif pada kedua mode; A menjadi kontrol HPA, bukan bukti efikasi ArgoCD memulihkan replika yang diabaikan. A, uji batas visibilitas F, dan kasus deskriptif G dilaporkan terpisah dari estimasi B–E. Tidak adanya OutOfSync bukan waktu deteksi nol. R3 tidak diukur karena audit API server tidak tersedia dan tidak dilaporkan sebagai nol atau tidak ada perbedaan. Penyesuaian tersebut tetap draf untuk tinjauan pemilik dan pembimbing; eksekusi formal dinonaktifkan.

## 1.3 Batasan Masalah

Ruang lingkup penelitian dibatasi sebagai berikut.

1. Perbandingan mencakup satu prosedur imperatif terotomasi berbasis `kubectl` dan satu implementasi GitOps berbasis ArgoCD. Tidak ada intervensi manusia dalam waktu deteksi atau pemulihan yang diukur; hasil tidak mewakili kecepatan operator atau semua implementasi kedua paradigma.

2. Beban uji formal adalah satu stack InvenioRDM multi-service pada satu klaster Kubernetes di namespace `invenio`. Tidak ada stack kembar `exp-*` dan tidak ada eksekusi paralel A/B. Karena stack yang sama dipakai bergantian, non-isolasi terhadap beban latar dan dependensi bersama menjadi keterbatasan eksplisit. Generalisasi dibatasi pada konfigurasi apparatus yang dilaporkan.

3. Surrogat S1 dan S2 merupakan artefak historis/offline untuk validasi instrumen. Keduanya tidak dipakai dalam desain formal, tidak digabungkan dengan data formal, dan tidak mendukung kesimpulan tentang InvenioRDM.

4. Intervensi yang direncanakan meliputi perubahan replika (A), image (B), ConfigMap (C), penghapusan sumber daya (D), patch requests/limits (E), sumber daya pada namespace di luar cakupan aplikasi (F), dan anotasi metadata Secret (G). A adalah kontrol HPA; F adalah batas visibilitas sumber daya tak terkelola; G bukan kasus *self-heal* ArgoCD dan dianalisis deskriptif.

5. R1 menilai kesetaraan inventaris dan bidang deklaratif yang ditetapkan dengan aturan sama pada kedua mode; kesehatan dilaporkan terpisah. R2 mencakup waktu deteksi, ketersediaan bukti penyimpangan, dan pemulihan konfigurasi penuh. R3 tidak diukur karena audit API server tidak tersedia; R3 dibahas sebagai keterbatasan dan pekerjaan lanjutan, bukan sebagai nol atau tidak ada perbedaan. Nilai Secret tidak dipublikasikan.

6. Analisis dibatasi pada hasil per skenario. Sampel formal adalah 15 pengulangan per skenario per mode (10 R2 dan 5 R1; total 210 run); angka tersebut bukan jaminan kecukupan daya. Data tersensor, kegagalan injeksi, dan kegagalan instrumentasi dilaporkan terpisah; tidak ada nilai hilang yang diganti nol.

## 1.4 Tujuan Penelitian

Penelitian ini bertujuan merancang dan, setelah protokol disetujui serta apparatus divalidasi, mengevaluasi perbedaan operasional antara mode imperatif terotomasi berbasis `kubectl` dan mode GitOps berbasis ArgoCD pada beban uji formal multi-service InvenioRDM. Evaluasi mengoperasionalkan RQ1 melalui kesetaraan konfigurasi dan kesehatan yang terpisah serta RQ2 melalui waktu deteksi, bukti, dan pemulihan konfigurasi. R3 tidak diukur. Tujuan ini tidak menyatakan sebelumnya mode mana yang lebih baik.

## 1.5 Manfaat Penelitian

Jika protokol berhasil divalidasi dan data dikumpulkan, penelitian ini diharapkan dapat:

1. memberikan deskripsi empiris yang dibatasi pada *workload*, skenario, dan konfigurasi yang benar-benar diuji;
2. mendokumentasikan definisi pengukuran serta aturan penanganan data tersensor agar dapat ditinjau dan direplikasi; dan
3. menyediakan informasi kontekstual bagi praktisi tanpa mengubah hasil satu kasus menjadi rekomendasi universal tentang GitOps atau pengelolaan imperatif.

## 1.6 Sistematika Penulisan

Skripsi ini disusun sebagai berikut:

- Bab I menjelaskan latar belakang, rumusan masalah, batasan, tujuan, manfaat, dan sistematika penulisan.
- Bab II membahas konsep drift, pendekatan pengelolaan konfigurasi, studi empiris terkait, serta ruang kontribusi yang dibatasi.
- Bab III menjelaskan rancangan perbandingan, *workload*, skenario, definisi metrik, protokol, analisis, dan ancaman validitas.
- Bab IV menyediakan struktur pelaporan setup dan bukti untuk R1–R2 serta batas pengukuran R3; hasil empiris hanya dapat diisi setelah eksperimen formal tervalidasi.

- Bab V merangkum status kesimpulan, keterbatasan, serta rekomendasi yang dapat ditindaklanjuti setelah hasil tersedia.

# BAB II

# TINJAUAN PUSTAKA

## 2.1 Penyimpangan Konfigurasi (*Configuration Drift*)

Penyimpangan konfigurasi (*configuration drift*) adalah ketidakcocokan antara keadaan aktual sistem dan keadaan yang ditetapkan sebagai acuan deklaratif (Pohjola, 2025). Drift berbeda dari cacat statis: konfigurasi yang tidak aman dapat tetap cocok dengan acuannya, sedangkan perubahan aktual yang tidak cocok dengannya merupakan drift meskipun nilai aktual belum dinilai dampaknya. Penelitian ini mengukur kesetaraan terhadap deklarasi, bukan kebenaran semantik atau keamanan seluruh manifest.

Penelitian empiris oleh Zhang et al. (2026) melakukan studi terhadap 719 *defect* konfigurasi dari 2.260 skrip Kubernetes dan mengidentifikasi 15 kategori *defect*, di mana 533 dari 719 *defect* (74,1%) menyebabkan *crash*, operasi yang salah, atau *outage*. Studi tersebut menegaskan bahwa *misconfiguration* merupakan risiko operasional utama pada Kubernetes. Rahman et al. (2023) menambahkan bahwa *security misconfiguration* pada manifest Kubernetes *open-source* mencakup 11 kategori yang berulang, termasuk *container* yang berjalan sebagai *root*, tanpa *drop capabilities*, dan tanpa *resource limits*. Penelitian menunjukkan bahwa penyimpangan konfigurasi bukan hanya masalah estetika konfigurasi, tetapi ancaman nyata terhadap keandalan dan keamanan layanan.

Pohjola (2025) memberikan kerangka konseptual untuk memahami sumber dan dampak penyimpangan pada sistem *Infrastructure-as-Code* (IaC). Penelitian tersebut mengidentifikasi tiga faktor pendorong utama penyimpangan: (1) keahlian manusia (*human expertise*) yang tidak konsisten; (2) tekanan waktu (*time pressure*) yang memicu perubahan cepat dan tidak didokumentasikan; dan (3) ketiadaan mekanisme deteksi otomatis pada *pipeline deployment*. Meskipun penelitian tersebut tidak beroperasi pada Kubernetes secara eksklusif, prinsip-prinsip yang dikemukakan berlaku umum untuk platform orkestrasi berbasis konfigurasi deklaratif.

### 2.1.1 Faktor yang Menimbulkan Penyimpangan Konfigurasi

Berdasarkan literatur, faktor-faktor utama yang memperburuk penyimpangan konfigurasi pada Kubernetes meliputi:

1. Perubahan langsung pada klaster: Penggunaan `kubectl edit`, `kubectl patch`, atau `kubectl scale` secara langsung tanpa memperbarui repositori *source of truth* (Zhang et al., 2026).

2. Prosedur *deployment* yang tidak terdokumentasi: Setiap *operator* dapat memiliki urutan perintah yang berbeda, yang mengakibatkan *state* klaster tidak deterministik (Pohjola, 2025).

3. Kurangnya mekanisme *rollback* yang terintegrasi: kemampuan untuk mengembalikan perubahan bergantung pada pencatatan versi, ketersediaan nilai dasar, dan fitur yang dikonfigurasi pada alat pengelolaan (Rahman et al., 2023).

4. Ketiadaan mekanisme deteksi penyimpangan otomatis: Tanpa pemeriksaan berkala antara *state* aktual dan *desired state*, penyimpangan dapat terakumulasi selama periode lama tanpa terdeteksi (Zhang et al., 2026).

## 2.2 Terminologi Dasar

Bagian ini mendefinisikan terminologi kunci yang digunakan dalam penelitian, dengan tujuan membangun dasar konseptual yang konsisten sebelum pembahasan lebih lanjut.

### 2.2.1 Kubernetes dan Orkestrasi Kontainer

Kubernetes adalah platform *container orchestration open-source* yang mengotomasi *deployment*, *scaling*, dan manajemen aplikasi terkemas (*containerized*) (Burns et al., 2016). Kubernetes awalnya dikembangkan oleh Google berdasarkan pengalaman mengelola sistem *workload* berskala besar di Borg dan Omega, dan kini menjadi standar de facto untuk orkestrasi kontainer di industri (Verma et al., 2015). Pada Kubernetes, *state* yang diinginkan (*desired state*) dideklarasikan melalui berkas konfigurasi (*manifest*), dan *control plane* bertanggung jawab mencapai dan mempertahankan *state* tersebut.

### 2.2.2 Git dan *Version Control*

Git adalah sistem *version control* terdistribusi yang merekam setiap perubahan pada berkas beserta metadata (penulis, *timestamp*, pesan *commit*) (Chacon and Straub, 2014). Dalam konteks GitOps, Git berfungsi sebagai *single source of truth*

yang menyimpan deklarasi *state* infrastruktur dan aplikasi, sehingga setiap perubahan dapat dilacak, di-*review*, dan dikembalikan (*rollback*) secara andal.

### 2.2.3 *Continuous Integration* dan *Continuous Deployment*

*Continuous Integration* (CI) adalah praktik di mana perubahan kode secara otomatis diuji dan digabungkan ke *branch* utama, sementara *Continuous Deployment* (CD) adalah perluasan CI di mana perubahan yang lulus pengujian secara otomatis di-*deploy* ke lingkungan produksi tanpa intervensi manual eksplisit (Humble and Farley, 2010). GitOps merupakan paradigma spesifik di dalam CD yang menggunakan repositori Git sebagai *single source of truth*.

### 2.2.4 *Infrastructure-as-Code* (IaC)

*Infrastructure-as-Code* (IaC) adalah praktik mendefinisikan dan mengelola infrastruktur melalui berkas konfigurasi yang diberi versi (*versioned*), bukan melalui proses manual (Morris, 2020). IaC memungkinkan *version control*, *code review*, dan reprodusibilitas lingkungan. Dalam konteks Kubernetes, IaC diimplementasikan melalui *manifest* deklaratif (YAML) yang mendefinisikan *desired state* klaster.

### 2.2.5 Paradigma Deklaratif vs Imperatif

Dalam paradigma imperatif, tindakan pengelolaan dinyatakan sebagai perintah atau urutan tindakan yang mengubah keadaan sistem (*how to achieve the state*). Dalam paradigma deklaratif, keadaan yang diinginkan dinyatakan dan pengendali bertugas mendekatinya (*what state to achieve*) (Scott, 2015). Keduanya dapat diotomasi; perbedaan untuk penelitian ini adalah prosedur koreksi imperatif tetap yang dijalankan melalui `kubectl` dibandingkan dengan rekonsiliasi ArgoCD.

## 2.3 Manajemen Konfigurasi Infrastruktur

*Infrastructure state management* mengatur bagaimana keadaan infrastruktur didefinisikan, diterapkan, dan dipertahankan. Pendekatan deklaratif menyimpan keadaan yang diinginkan dalam konfigurasi, sedangkan pendekatan imperatif menyatakan tindakan untuk mengubah keadaan (Morris, 2020; Limoncelli et al., 2018). Otomasi, pemeriksaan berulang, dan pencatatan dapat tersedia pada kedua pendekatan; karakteristik tersebut perlu ditetapkan pada tingkat implementasi yang dibandingkan.

## 2.4 *Continuous Deployment* dan Paradigma GitOps

*Continuous Deployment* (CD) adalah praktik dalam ranah DevOps di mana setiap perubahan kode yang lulus pengujian secara otomatis di-*deploy* ke lingkungan produksi tanpa intervensi manual eksplisit (Limoncelli et al., 2018). CD merupakan perluasan dari *Continuous Integration* (CI) dan bertujuan meminimalkan waktu antara penulisan kode dan ketersediaannya bagi pengguna akhir. Dalam ekosistem Kubernetes, CD memerlukan mekanisme yang dapat secara andal menyelaraskan perubahan konfigurasi dengan *state* klaster.

GitOps adalah paradigma spesifik di dalam CD yang menggunakan repositori Git sebagai *single source of truth* untuk mendefinisikan *state* infrastruktur dan aplikasi. CNCF GitOps Working Group mendefinisikan empat prinsip OpenGitOps: (1) sistem dideklarasikan secara deklaratif; (2) *state* sistem yang diinginkan tersimpan dan diberi versi (*versioned*) dalam Git; (3) perubahan *state* ditarik (*pulled*) secara otomatis oleh agen *software*; dan (4) agen *software* secara kontinu mendeteksi dan mengoreksi deviasi dari *state* yang diinginkan (CNCF GitOps Working Group, 2021).

GitOps mendukung pencatatan versi deklarasi melalui Git, peninjauan perubahan, dan penggunaan revisi sebelumnya sebagai acuan pemulihan (Yuen et al., 2021). Riwayat Git tidak otomatis merekam perubahan langsung pada klaster; tindakan tersebut memerlukan bukti dari pengendali atau audit API. Pada model *pull-based*, agen di dalam klaster dapat menarik deklarasi tanpa pipeline eksternal menerapkan setiap perubahan ke API klaster, tetapi kebutuhan kredensial dan akses operasional tetap bergantung pada rancangan serta konfigurasi sistem (Utami and Fatta, 2021). Klaim adopsi dan manfaat GitOps juga muncul dalam artikel industri CNCF 2025; artikel tersebut dipakai hanya sebagai konteks industri, bukan sebagai bukti *peer-review* (Lopez and Narang, 2025).

### 2.4.1 Performa *Deployment* dan *Anti-Pattern*

Forsgren dkk. (Forsgren et al., 2018) membahas *deployment frequency*, *lead time for changes*, *time to restore service*, dan *change failure rate* sebagai metrik performa pengembangan dan operasi. Metrik organisasi tersebut tidak identik dengan pengukuran eksperimen ini dan tidak dengan sendirinya membuktikan bahwa satu prosedur selalu lebih unggul. Penelitian ini menggunakan konstruk yang lebih terbatas: kesetaraan bidang konfigurasi, waktu deteksi dan pemulihan konfigurasi, serta bukti keterlacakan.

ArgoCD menyediakan pengendali aplikasi yang membandingkan sumber daya teramati dengan deklarasi repositori dan, jika kebijakan serta cakupannya mengizinkan, dapat melakukan sinkronisasi atau koreksi otomatis (Argo Project, 2024). Perilaku aktual bergantung pada konfigurasi aplikasi, izin, ketersediaan pengendali, jenis sumber daya, dan cakupan inventaris. Oleh sebab itu, rekonsiliasi tidak menjamin setiap perubahan akan selalu ditemukan atau dipulihkan.

## 2.5 Dasar Teori: *Reconciliation* dan *Convergence* pada Sistem Deklaratif

Karakteristik GitOps yang berbeda dari pendekatan imperatif dapat dijelaskan melalui dua prinsip teoretis yang saling berkaitan.

### 2.5.1 *Convergence* Menuju *Desired State*

Pengendali deklaratif secara berkala membandingkan keadaan aktual dengan keadaan yang diinginkan dan dapat melakukan tindakan korektif sesuai kebijakan yang berlaku (Argo Project, 2024). Keberhasilan konvergensi bukan sifat yang dapat diasumsikan untuk semua sumber daya atau gangguan: hasil dipengaruhi oleh cakupan aplikasi, izin, ketersediaan komponen, konfigurasi sinkronisasi, dan sifat sumber daya.

Perintah imperatif tidak dengan sendirinya menyediakan rekonsiliasi berkelanjutan. Namun, prosedur imperatif dapat diotomasi dengan pemeriksaan berkala dan tindakan koreksi yang telah ditentukan. Karena penelitian ini membandingkan prosedur `kubectl` terotomasi dengan ArgoCD, bukan respons manusia, perbedaan waktu tidak ditafsirkan sebagai perbedaan kecepatan operator.

### 2.5.2 *Audit Trail* sebagai *Social Control Mechanism*

Riwayat Git dapat menyediakan metadata dan perubahan deklarasi, sedangkan log pengendali dan audit API dapat memberi bukti tambahan tentang penerapan aktual (CNCF GitOps Working Group, 2021). Riwayat tersebut tidak otomatis membuktikan siapa yang melakukan perubahan pada klaster atau apakah semua tindakan terekam; kelengkapan bergantung pada tata kelola repositori, identitas, konfigurasi audit, dan pengumpulan bukti. Namun, R3 tidak diukur dalam penelitian ini karena audit API server Kubernetes tidak tersedia; keterlacakan dibahas sebagai keterbatasan dan pekerjaan lanjutan, bukan sebagai hasil.

## 2.6 *Runbook* sebagai Standarisasi Prosedur Pemulihan

*Runbook* adalah dokumen terstandarisasi yang berisi prosedur operasional untuk menanggapi insiden atau melakukan pemulihan (*recovery*) pada sebuah sistem. Dalam ranah operasi TI, *runbook* berfungsi sebagai *playbook* yang mendefinisikan langkah-langkah korektif berurutan berdasarkan jenis insiden, sehingga setiap *operator* yang berbeda mengikuti urutan tindakan yang sama (Schlette et al., 2024). Penggunaan *runbook* mengurangi variabilitas akibat keahlian *operator* yang berbeda-beda dan memastikan bahwa keputusan subjektif diminimalkan selama pemulihan.

Dalam rancangan penelitian ini, prosedur imperatif bukan runbook yang dipandu kepada operator. Mode imperatif menggunakan pemeriksa dan korektor otomatis yang tetap, dengan pembandingan deklarasi dan keadaan aktual serta tindakan `kubectl` yang telah ditetapkan. Mode ini diperoleh dengan menjeda *auto-sync*/*self-heal* ArgoCD pada stack yang sama, lalu memulihkannya. Pilot perlu memverifikasi bahwa prosedur tersebut bekerja tanpa menerima identitas skenario atau label injeksi dan menyimpan bukti yang sama jenisnya dengan evaluator bersama.

## 2.7 Paradigma *Deployment* Alternatif

Pilihan pengelolaan konfigurasi mencakup rekonsiliasi berbasis tarik, pipeline dorong, alat konfigurasi yang dijalankan sesuai kebutuhan, serta otomasi imperatif khusus. Perbedaan autentikasi, cakupan sumber daya, pencatatan, dan frekuensi eksekusi bergantung pada konfigurasi implementasi; tidak diasumsikan bahwa seluruh pipeline dorong mengekspos kredensial atau bahwa semua alat imperatif tidak memiliki audit. Studi Shrestha dan Ali secara khusus membandingkan GitOps dengan Ansible pada waktu induksi, penemuan, dan pemulihan drift (Shrestha and Ali, 2024).

Penelitian ini membatasi pembanding pada satu prosedur imperatif terotomasi berbasis `kubectl` dan satu instalasi ArgoCD. Kontras ini tidak merupakan batas atas efek yang berlaku bagi semua alat atau konteks; temuan nantinya hanya berlaku bagi implementasi dan workload yang benar-benar diuji.

## 2.8 Evaluasi Empiris GitOps dan ArgoCD

Studi terkait telah menguji sejumlah kombinasi alat dan tujuan penelitian. Shrestha dan Ali (Shrestha and Ali, 2024) membandingkan GitOps dengan Ansible pada waktu induksi, penemuan, dan pemulihan drift. Kagganti Nataraja (Nataraja,

2025) mendeskripsikan perbandingan terkontrol pada klaster Minikube satu node dengan konfigurasi keamanan yang berbeda. Matubber (Matubber, 2025) mengevaluasi pengelolaan infrastruktur Kubernetes berbasis GitOps dan mencakup pembanding lama yang semi-terotomasi. Ketiganya menunjukkan bahwa penelitian komparatif atau evaluasi berbasis waktu telah ada; abstrak yang tersedia tidak cukup untuk menyimpulkan ada atau tidaknya uji statistik tertentu.

Utami dan Al Fatta (Utami and Fatta, 2021) membahas model deployment deklaratif berbasis tarik dibandingkan dengan model berbasis dorong. Paavola (Paavola, 2021) membahas pengelolaan beberapa aplikasi Kubernetes menggunakan prinsip GitOps, sedangkan D'Amore (D'Amore, 2021) mendeskripsikan implementasi ArgoCD pada lingkungan Kubernetes hybrid cloud. Studi tersebut memberi konteks implementasi, tetapi tidak diperlakukan di sini sebagai bukti bahwa salah satu prosedur selalu lebih unggul.

Perbandingan penelitian perlu dibaca sesuai tujuan, alat, lingkungan, konfigurasi, dan hasil yang benar-benar dilaporkan oleh masing-masing sumber. Draf ini tidak menyatakan bahwa penelitian terdahulu tidak terkontrol, tidak menggunakan statistik, atau tidak memiliki baseline apabila pernyataan itu tidak dapat dipastikan dari abstrak yang tersedia.

## 2.9 Metode Eksperimental Terkontrol pada Riset Sistem

Evaluasi empiris pada riset sistem dan infrastruktur memerlukan desain yang memitigasi ancaman terhadap validitas (Wohlin et al., 2012). Penelitian ini merupakan studi kasus komparatif terkontrol: satu kasus diamati pada kondisi dasar, satu intervensi, dan kondisi pascaintervensi, dengan dua mode pengelolaan yang dijalankan berurutan (Runeson and Höst, 2009). Limoncelli et al. (2018) menjelaskan bahwa penelitian sistem harus membangun lingkungan yang dapat direproduksi, mengendalikan variabel pengganggu, dan mendefinisikan metrik yang jelas sebelum menyimpulkan hubungan kausal.

Rancangan penelitian ini menetapkan mode imperatif terotomasi berbasis `kubectl` dan mode GitOps berbasis ArgoCD sebagai dua perlakuan pada satu stack. Kesetaraan *workload*, nilai dasar, inventaris pengukuran, dan prosedur injeksi direncanakan untuk mengurangi perbedaan yang tidak relevan. Karena kedua implementasi dapat berbeda dalam lebih dari satu komponen teknis dan tidak ada isolasi penuh, hasil tidak langsung diatribusikan sebagai efek universal paradigma; interpretasinya

dibatasi pada prosedur dan kasus yang benar-benar diuji (Wohlin et al., 2012).

Analisis yang direncanakan bersifat per skenario dan eksploratif. Ringkasan distribusi, jumlah observasi lengkap dan tersensor, serta ukuran efek dan interval kepercayaan dilaporkan bersama-sama; uji komparatif hanya digunakan jika asumsi dan struktur data mendukungnya. Sampel formal 15 pengulangan per skenario per mode (210 run) tidak dianggap sebagai bukti daya statistik yang memadai (Dybå et al., 2006; Kitchenham and Madeyski, 2024). Rincian definisi operasional dan batas interpretasi terdapat pada Bab III.

## 2.10 Klasifikasi Beban Uji (*Workload*)

Aplikasi yang dijalankan pada klaster Kubernetes dapat dikelompokkan ke dalam beberapa kelas *workload* berdasarkan karakteristik arsitektur dan manajemen *state* (Siddiqui et al., 2023; Aghili et al., 2024):

1. *Workload* web tanpa state (*stateless web microservice*): Layanan yang tidak menyimpan *state* secara internal, sehingga setiap permintaan dapat dilayani oleh instans mana pun. Contoh: server HTTP sederhana, *API gateway*.

2. *Workload* berbasis data yang menyimpan *state* (*stateful data service*): Layanan yang menyimpan *state* secara persisten, seperti *database* relasional, *search engine*, atau *object storage*. *Workload* kelas ini memerlukan urutan *deployment* dan penanganan penyimpangan yang lebih kompleks.

3. *Workload* komposit multi-service: Aplikasi yang terdiri atas banyak komponen yang saling bergantung, mencakup kombinasi kelas 1 dan 2 beserta komponen infrastruktur (*ingress*, TLS, *secrets*, *monitoring*, *backup*, kebijakan keamanan). Kelas *workload* ini memiliki kompleksitas dependensi tertinggi dan paling rentan terhadap penyimpangan konfigurasi akibat banyaknya komponen yang harus diselaraskan.

Pengujian formal dalam rancangan ini menggunakan beban uji komposit multi-service InvenioRDM pada satu stack produksi-like. Inventaris dan komponen aktual harus dicatat dari konfigurasi yang divalidasi sebelum eksperimen; jumlah komponen yang pernah disebut sebagai target belum diverifikasi. Beban uji merupakan konteks pengukuran, bukan dasar untuk menggeneralisasi hasil ke seluruh *workload* multi-service. Surrogat S1/S2 historis/offline tidak dipakai dalam desain formal dan tidak memperluas sampel formal.

## 2.11 Ruang Kontribusi Penelitian

Studi komparatif mengenai GitOps dan pengelolaan konfigurasi telah ada, termasuk evaluasi terhadap Ansible, pengujian terkontrol pada Minikube, dan pengukuran yang mencakup *baseline* semi-terotomasi. Karena itu, penelitian ini tidak berangkat dari klaim bahwa perbandingan terdahulu tidak ada atau bahwa keunggulan GitOps telah terbukti secara universal.

Kontribusi yang diklaim bersifat terbatas dan dapat diperiksa: (1) perbandingan empiris pada *deployment* InvenioRDM multi-service nyata, bukan *workload* sintetis; (2) dua mode yang dijalankan berurutan pada satu stack produksi-like, dengan mode imperatif didefinisikan sebagai penjedaan ArgoCD ditambah pemeriksa/*runbook* `kubectl`, sehingga perbandingan tidak memerlukan stack kembar; (3) pengukuran R1/R2 yang dapat direproduksi dengan kepemilikan HPA, batas visibilitas F, dan kasus G yang dipisahkan dari efikasi; serta (4) bukti batas bahwa hasilnya berlaku untuk kasus dan apparatus yang diuji, bukan klaim superioritas GitOps universal. R3 tidak diukur karena audit API server tidak tersedia dan dibahas sebagai keterbatasan serta pekerjaan lanjutan.

Hasil yang kelak diperoleh hanya dapat menjawab pertanyaan pada skenario, *workload*, konfigurasi, dan resolusi instrumentasi yang benar-benar diuji. Besaran maupun arah perbedaan belum dapat dipastikan sebelum pengumpulan serta analisis data formal.

# BAB III

# METODE PENELITIAN

## 3.1 Deskripsi Umum dan Status Protokol

Penelitian ini dirancang sebagai studi kasus komparatif terkontrol (*controlled single-case comparative study*) atas dua mode pengelolaan pada satu stack InvenioRDM produksi-like: mode imperatif terotomasi berbasis `kubectl` dan mode GitOps berbasis ArgoCD. Pertanyaan yang dituju ialah bagaimana kedua mode berbeda pada konsistensi konfigurasi serta waktu deteksi, ketersediaan bukti penyimpangan, dan pemulihan konfigurasi untuk skenario yang ditetapkan. Estimand dibatasi pada selisih hasil antarmode dalam konteks beban uji InvenioRDM dan apparatus eksperimen yang diverifikasi; estimand ini bukan efek universal seluruh paradigma imperatif atau GitOps.

Kedua mode bukan dua *deployment* terpisah. Pada mode imperatif, *auto-sync* dan *self-heal* ArgoCD dijeda, sedangkan pemeriksa dan *runbook* otomatis tetap bekerja dengan `kubectl`; setelah blok selesai, ArgoCD dipulihkan dan diverifikasi. Pada mode GitOps, ArgoCD aktif. Tidak ada eksekusi paralel A/B, tidak ada stack kembar `exp-*`, dan tidak ada operator yang menunggu, mengetik, atau memilih tindakan korektif selama pengukuran. Karena satu stack dipakai bergantian, perlakuan tidak terisolasi dari beban latar, dependensi bersama, atau keadaan klaster yang berubah; non-isolasi tersebut merupakan keterbatasan eksplisit.

Naskah ini memuat rancangan protokol, bukan laporan pelaksanaan. Protokol masih memerlukan tinjauan pembimbing, dan apparatus eksperimen yang tersedia belum divalidasi untuk mengimplementasikan seluruh definisi operasional di bawah. Versi perangkat lunak, kesiapan klaster, inventaris sumber daya formal, dan data eksperimen belum ditetapkan atau diverifikasi dalam naskah ini. Pilot terbatas pada stack yang sama bukan data formal. Karena itu, uraian rencana tidak boleh dibaca sebagai bukti bahwa eksperimen telah dilaksanakan.

Penyesuaian kepemilikan pengendali tetap berstatus draf untuk tinjauan pemilik penelitian dan pembimbing: HPA normal aktif pada kedua mode dan skenario A menjadi kontrol respons HPA. A tidak digabungkan ke efikasi ArgoCD pada B–E; F adalah kasus batas visibilitas tersendiri; G adalah kasus deskriptif yang bukan *self-heal* ArgoCD. Protokol telah diimplementasikan pada harness live, dan pengum-

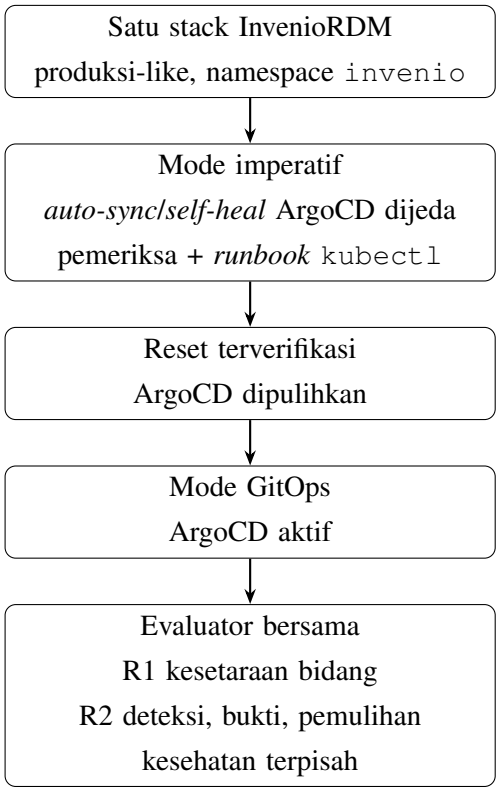
pulan formal dimulai pada 2026-10-27 pada satu stack produksi-like; hasil numerik belum dianalisis dan tidak dilaporkan di sini. Instrumen live menjalankan baseline, injeksi, pemulihan, dan reset dengan gerbang kegagalan tertutup; verifikasi seed memakai token dan tidak menulis nilai Secret. Audit API server tidak tersedia; ini tidak menghambat *deployment*, tetapi menutup R3.

## 3.2 Rancangan Perbandingan

Pengendalian kondisi awal, pengulangan, pencatatan penyimpangan protokol, dan pembatasan inferensi mengikuti prinsip studi kasus dan evaluasi empiris perangkat lunak (Wohlin et al., 2012; Runeson and Höst, 2009). Karena kasus tidak dipilih acak dari populasi dan kedua mode berurutan pada stack yang sama, rancangan tidak dianggap sebagai penugasan acak klaster dari suatu populasi (Cook and Campbell, 1979).

Unit analisis adalah satu injeksi penyimpangan yang diterima pada satu mode dalam satu pengulangan. Mode imperatif menggunakan otomasi tetap: pemeriksa berkala membandingkan keadaan aktual dengan inventaris dan nilai yang diinginkan, menyimpan perbedaan yang teramati, lalu menjalankan prosedur koreksi `kubectl` yang telah ditentukan. Kepemilikan rekonsiliasi ArgoCD dijeda secara terkendali dengan menonaktifkan *auto-sync*/*self-heal* pada `apps` dan `invenio-bootstrap`, lalu dipulihkan pada blok `finally` dan diverifikasi (Argo Project, 2025). Mode GitOps menggunakan ArgoCD dengan rekonsiliasi otomatis terhadap deklarasi Git. Tidak ada operator yang menunggu, mengetik, atau memilih tindakan korektif selama pengukuran; waktu respons manusia maupun tingkat keahlian operator bukan variabel penelitian.

Kedua mode memakai beban uji, nilai dasar, definisi bidang yang diperiksa, serta prosedur injeksi dan pengukuran yang setara. Injeksi dilakukan satu per satu, bukan serentak, dan urutan mode serta skenario diacak dalam blok pengujian setelah kondisi dasar dipulihkan. Pengendali koreksi hanya menerima keadaan aktual dan deklarasi yang diperlukan untuk membandingkannya; identitas skenario tidak diteruskan sebagai label, termasuk pada skenario Secret. Rancangan ini membandingkan dua implementasi operasional yang telah ditentukan, bukan mengisolasi seluruh faktor yang mungkin membedakan setiap produk atau organisasi.

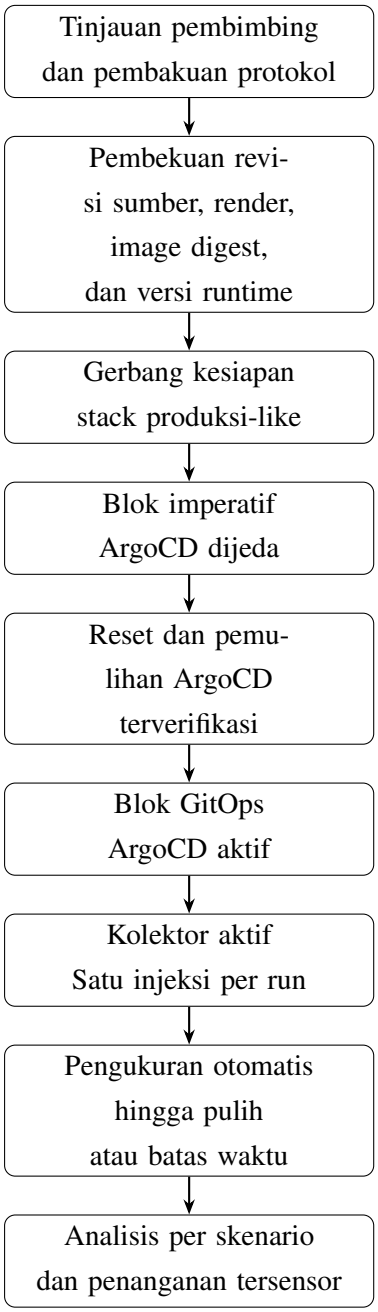



Gambar 3.1: Rancangan dua mode pengelolaan berurutan pada satu stack InvenioRDM; mode imperatif adalah penjedaan ArgoCD, bukan *deployment* terpisah.

Gambar 3.1 menunjukkan bahwa injeksi, mekanisme pengelolaan, dan evaluator merupakan bagian yang dibedakan secara eksplisit. Hasil hanya akan ditafsirkan untuk konfigurasi apparatus yang benar-benar dicatat saat eksperimen; HPA adalah pengendali bersama pada kedua mode.

### 3.3 Alur Penelitian

Tahapan yang direncanakan meliputi peninjauan protokol, pembekuan sumber dan versi, validasi instrumen offline, pengujian formal, dan analisis per skenario. Surrogat S1/S2 merupakan artefak historis/offline; keduanya tidak dipakai dalam desain formal dan tidak masuk ke sampel formal.




Gambar 3.2: Bagan alir protokol dengan pembekuan sumber sebelum blok pengukuran dan tanpa surrogat pada desain formal.

Gambar 3.2 menempatkan pembekuan sumber dan validasi sebelum pengumpulan data formal serta mensyaratkan pemulihan kondisi dasar di antara pengulangan. Tidak ada jeda buatan setelah injeksi untuk menunggu respons manusia.

## 3.4 Beban Uji dan Lingkungan

Pengujian formal menggunakan beban uji komposit multi-service InvenioRDM dengan sasaran sedikitnya 16 komponen aplikasi dan infrastruktur pendukung sesuai ruang lingkup proposal. Sasaran tersebut bukan jumlah komponen yang telah diverifikasi. Inventaris formal mencakup seluruh sumber daya dalam manifest yang disetujui; daftar komponen, batas pengelolaan, dan versinya dicatat dari artefak yang benar-benar digunakan. S1/S2 merupakan surrogat historis/offline, bukan pengganti beban formal, dan tidak boleh digabungkan ke hasil atau analisis formal.

Rancangan membatasi pengujian pada satu klaster Kubernetes, satu stack InvenioRDM produksi-like di namespace `invenio`, dan satu implementasi GitOps, yaitu ArgoCD. Tidak ada namespace kembar `exp-*`, tidak ada *deployment* kembar, dan tidak ada eksekusi paralel. Spesifikasi serta versi aktual klaster, node, komponen beban uji, dan ArgoCD harus diverifikasi dan dicatat sebelum pengujian; nilai yang pernah disebut sebagai sasaran pada draf sebelumnya belum menjadi fakta eksperimen. Karena satu stack dipakai bergantian, komponen infrastruktur bersama tidak dapat diisolasi dari perlakuan; injeksi dan koreksi dibatasi pada objek dalam kontrak bidang.

Konteks *deployment* yang dilaporkan pemilik adalah Kubernetes 1.32.6/RKE2, containerd, dan Ubuntu 24.04. Penunjuk sumber otoritatif ada pada `experiments/deployment-source.json`, dengan revisi referensi `6cacea6642a7c1690243d3b54130fa08216a9595`. Application `argocd/apps/invenio-bootstrap.yaml` memilih `k8s/apps/invenio/kustomization.yaml`; konfigurasi sumber rekonsiliasi menetapkan 30 detik, bukan target lama 60 detik. Revisi referensi bukan bukti revisi yang diterapkan. Sebelum formal, rekam revisi runtime aktual, render dan digest-nya, image digest, inventaris, serta interval pengendali; tidak dibuat sumber manifest editable kedua di repositori penelitian.

### 3.4.1 Gerbang Kesiapan

Pengulangan formal hanya dimulai setelah seluruh prasyarat berikut diperiksa dan dicatat: (1) manifest dan inventaris sumber daya formal telah dibekukan; (2) stack berada pada kesetaraan nilai yang ditetapkan dan kesehatan layanan telah memenuhi kriteria kesiapan; (3) *auto-sync*, *self-heal*, cakupan aplikasi, dan kebijakan prune ArgoCD telah diverifikasi, termasuk prosedur penjedaan dan pemulihan untuk mode

imperatif; (4) pemeriksa imperatif, pengumpul bukti, serta pembanding bidang telah lulus validasi offline; (5) jam pencatat waktu tersinkronisasi; dan (6) tidak ada pengujian atau perubahan lain yang berlangsung pada lingkup sumber daya yang diamati. Ambang kesehatan, batas beban, interval ArgoCD, serta batas waktu pengamatan dibakukan setelah pilot dan tinjauan pembimbing; naskah ini tidak menyatakan nilai aktualnya.

Jika prasyarat gagal, injeksi tidak dimulai. Gangguan setelah injeksi dicatat sebagai kejadian pengujian, bukan dihapus tanpa jejak. Kriteria penghentian, nilai batas waktu, dan aturan pengulangan harus ditetapkan sebelum data formal dikumpulkan.

Gerbang CPU dan memori tetap ketat di bawah 70% pada setiap node; memori memakai *working set* metrics-server dibagi *allocatable*. Bukti metrik, inventaris, resource requests/limits, pod Running dan Ready, serta reset aplikasi wajib lengkap. Audit API tidak tersedia; ini tidak menjadi gerbang *deployment*, tetapi menutup R3. Kondisi worker02 sekitar 71% yang dilaporkan pemilik, SMTP placeholder, kapasitas satu stack, serta *private-draft seed*/recovery yang tertunda tidak dianggap teratasi. Bootstrap/migrasi/fixtures/indexing/queue tidak boleh dipicu sebagai bagian repair rutin tanpa desain lifecycle yang ditinjau. Interval rekonsiliasi repositori, *watch* kejadian, *retry self-heal*, dan evaluator dibedakan.

### 3.5 Intervensi dan Skenario

Intervensi adalah satu perubahan terkontrol terhadap keadaan sumber daya dalam satu pengulangan. Skenario A–E menyasar sumber daya yang termasuk inventaris aplikasi; F menguji batas visibilitas untuk sumber daya di luar cakupan aplikasi; G adalah kasus deskriptif pada metadata Secret dan bukan *self-heal* ArgoCD.

Tabel 3.1: Skenario intervensi yang direncanakan

| Skenario | Intervensi | Batas interpretasi dan analisis |
|---|---|---|
| A | Mengubah jumlah replika Deployment yang dikendalikan HPA | Kontrol HPA normal pada kedua mode; `/spec/replicas` dikecualikan dari R1 statis; bukan koreksi ArgoCD |
| B | Mengubah referensi image Deployment terkelola | Drift terkelola; perubahan bidang yang telah ditetapkan |
| C | Mengubah bidang ConfigMap terkelola | Drift terkelola; tidak mengubah deklarasi sumber kebenaran |
| D | Menghapus sumber daya terkelola | Drift terkelola; pemulihan dinilai terhadap inventaris dan nilai dasar |
| E | Mengubah bidang sumber daya terkelola melalui patch | Drift terkelola; hanya bidang yang telah ditetapkan yang dinilai |
| F | Membuat sumber daya pada namespace di luar cakupan aplikasi | Batas visibilitas sumber daya tak terkelola; deskriptif, bukan keberhasilan atau kegagalan *self-heal* |
| G | Mengubah anotasi metadata Secret terkelola | Bukan kasus *self-heal* ArgoCD; pemulihan oleh harness; deskriptif dan tidak dipool |

Pada skenario A, `/spec/replicas` web/worker dimiliki HPA dan dikecualikan dari kesetaraan statis. Respons diukur terhadap keputusan HPA yang teramati, bukan angka tetap tiga replika. Tidak adanya OutOfSync tidak berarti deteksi nol; HPA dapat memulihkan replika tanpa sinyal ArgoCD. Hasil A dilaporkan sebagai kontrol HPA terpisah. Penghormatan kepemilikan replika saat *sync* harus diverifikasi, bukan diasumsikan dari *ignoreDifferences* saja.

Pada skenario F, sumber daya di luar inventaris aplikasi tidak dianggap sebagai drift terkelola yang wajib dipulihkan ArgoCD. Pengamatan dibatasi pada apakah batas cakupan dan pencatatan inventaris membuat keberadaan sumber daya tersebut terlihat. A, F, dan G tidak digabungkan ke estimasi utama B–E.

Pada skenario G, perubahan berupa anotasi metadata pada SealedSecret sinte-

tis; pemulihan anotasi dilakukan oleh harness, bukan oleh rekonsiliasi ArgoCD. Nilai `spec.encryptedData` tidak dibaca atau ditulis. G dianalisis secara deskriptif dan tidak dipool ke estimasi efikasi B–E. Pembekuan ImageUpdater dan pengendali SealedSecret memerlukan verifikasi sebelum formal.

## 3.6 Definisi Variabel dan Metrik

### 3.6.1 R1: Kesetaraan Konfigurasi dan Kesehatan

Kesetaraan konfigurasi dinilai menggunakan aturan yang sama pada kedua mode. Sebelum eksperimen, skema bidang yang diperiksa dan inventaris sumber daya yang diharapkan dibekukan. Bidang dinamis yang tidak relevan bagi deklarasi dikecualikan secara eksplisit. Untuk setiap sumber daya yang diharapkan, sumber daya dinyatakan cocok jika keberadaannya sesuai inventaris dan semua bidang terpilih sama dengan nilai yang diinginkan.

$$R1_{eq} = \frac{\text{jumlah sumber daya terkelola yang cocok}}{\text{jumlah sumber daya dalam inventaris yang diharapkan}} \times 100\%.$$

Evaluator menghitung $R1_{eq}$ dari pembacaan keadaan aktual dan inventaris yang sama bentuknya untuk kedua mode; status *Synced* ArgoCD bukan pengganti aturan kesetaraan tersebut. Nilai dicatat pada kondisi dasar, pengamatan pertama setelah injeksi, dan setelah prosedur koreksi. Jika koreksi mendahului pengamatan pascainjeksi, kondisi sesaat tersebut ditandai tidak teramati; kondisi drift tidak direkonstruksi seolah-olah telah diukur. Nilai Secret dibandingkan secara aman di memori; nilai mentah maupun hash yang dapat digunakan untuk menebak isinya tidak ditulis ke log, gambar, atau laporan.

Kontrak bidang mencakup image, requests/limits, ConfigMap, Secret dan bidang Service yang relevan. Bidang dinamis HPA tidak dibandingkan dengan nilai replika statis Git. Bidang yang tidak diperiksa atau kegagalan API menggagalkan bukti; sumber daya yang benar-benar tidak ada pada daftar lengkap memberi ketidakcocokan. Kesehatan *rollout* tidak menggantikan pemeriksaan bidang tersebut.

Kesehatan layanan dilaporkan sebagai hasil terpisah berdasarkan kriteria kesiapan yang ditetapkan sebelum eksperimen, misalnya kondisi kesiapan Pod dan status layanan yang memang tersedia pada *workload*. Kesehatan tidak dimasukkan ke pembilang kesetaraan konfigurasi. Sumber daya ekstra di luar inventaris, termasuk F,

dilaporkan terpisah dan tidak mengubah penyebut R1.

### 3.6.2 R2: Deteksi, Ketersediaan Bukti, dan Pemulihan

Semua waktu berasal dari satu kolektor pada host pencatat. Setiap kejadian diberi waktu jam monotonik dalam detik dengan pecahan desimal serta cap waktu UTC untuk audit; durasi dihitung dari jam monotonik, bukan dari selisih jam dinding. Pengamatan diaktifkan sebelum injeksi dan berlangsung pada interval nominal satu detik. Cap waktu adalah waktu pengamatan pertama, bukan waktu kejadian yang diinterpolasi; interval aktual dan jeda polling dicatat. Interval ini merupakan resolusi pengamatan yang direncanakan dan harus divalidasi pada pilot. Interval pemeriksa imperatif dan jadwal rekonsiliasi ArgoCD dicatat tersendiri dari interval evaluator, dibekukan sebelum pengujian formal, dan tidak disesuaikan setelah hasil terlihat.

- $t_0$: waktu kolektor mulai mengirim permintaan injeksi ke Kubernetes API. Waktu penerimaan respons API juga dicatat sebagai $t_a$, sehingga $t_a - t_0$ mendokumentasikan latensi permintaan. Injeksi diterima hanya jika operasi API berhasil dan objek respons atau pembacaan balik memverifikasi perubahan bidang sasaran. Durasi sejak $t_0$ mencakup latensi permintaan, bukan semata-mata waktu respons setelah perubahan tersimpan. Injeksi gagal atau tidak terverifikasi dicatat sebagai kegagalan injeksi, bukan sebagai waktu deteksi nol.

- $t_d$: waktu pertama mekanisme pengelolaan memberi sinyal deteksi berdasarkan perbedaan dari nilai yang diinginkan. Pada mode imperatif, sinyal berasal dari pemeriksa imperatif; pada mode GitOps, dari status perbedaan ArgoCD yang relevan untuk B–E. Tidak ada sinyal skenario tersembunyi yang diberikan kepada mekanisme tersebut. Sinyal HPA dicatat sebagai sinyal HPA, bukan deteksi ArgoCD.

- $t_e$: waktu pertama bukti dari mekanisme pengelolaan tersimpan dan mengidentifikasi sumber daya serta bidang yang berbeda terhadap acuan. Bukti harus dapat diperiksa ulang tanpa memakai label injeksi yang tersembunyi; untuk Secret cukup dicatat identitas bidang dan hasil ketidakcocokan, bukan isinya. Kriteria ini hanya mengukur bukti deviasi, bukan identifikasi penyebab. $t_e$ dapat sama dengan $t_d$, tetapi keduanya tetap dicatat terpisah. Pengumpulan bukti tidak menunda koreksi pada salah satu mode.

- $t_r$: waktu pengamatan pertama ketika inventaris dan seluruh bidang R1 kembali cocok dengan nilai yang diinginkan, dengan kecocokan dikonfirmasi pada pembacaan berikutnya. Waktu yang dilaporkan tetap waktu pengamatan pertama; konfirmasi hanya mencegah keadaan sesaat dinyatakan pulih. Kesehatan layanan dilaporkan terpisah.

Durasi utama adalah waktu deteksi $t_d - t_0$, waktu ketersediaan bukti $t_e - t_0$, dan waktu pemulihan konfigurasi penuh $t_r - t_0$. Selang dari deteksi ke bukti ($t_e - t_d$) dapat ditampilkan sebagai rincian tambahan. $t_r$ berarti pemulihan kesetaraan inventaris dan bidang yang diukur, bukan klaim bahwa seluruh layanan sehat atau tidak mengalami dampak lain.

Sinyal deteksi mode imperatif berasal dari pemeriksaan otomatis terhadap keadaan aktual; setelah sinyal muncul, prosedur koreksi tetap menerapkan deklarasi yang diinginkan menggunakan tindakan `kubectl` yang telah ditetapkan. Mode GitOps menggunakan rekonsiliasi ArgoCD. Kedua prosedur berjalan otomatis tanpa waktu reaksi manusia dan tanpa jeda buatan. Keputusan koreksi menggunakan hasil perbandingan umum, bukan label A–G yang disembunyikan dari responden.

### 3.6.3 R3: Batas Pengukuran

R3 (bukti keterlacakan) tidak diukur dalam penelitian ini karena audit API server Kubernetes tidak tersedia. Tanpa audit, pelaku mutasi *out-of-band* tidak dapat direkonstruksi, sehingga klaim keterlacakan tidak dapat dibuat untuk kedua mode. Ketiadaan audit tidak menghambat *deployment* atau pengukuran R1/R2; ia hanya menutup R3. R3 tidak dilaporkan sebagai nol, tidak dilaporkan sebagai tidak ada perbedaan, dan tidak diuji secara statistik. Unsur pelaku, objek dan bidang, waktu dan urutan, serta tindakan dan hasil didokumentasikan sebagai keterbatasan dan pekerjaan lanjutan setelah audit diaktifkan.

## 3.7 Protokol Pengulangan dan Penanganan Kegagalan

Satu pengulangan mengikuti urutan berikut:

1. Periksa gerbang kesiapan dan simpan bukti kondisi dasar, inventaris, kesehatan, serta status instrumentasi. Bekukan revisi sumber, SHA-256 render, image digest, dan versi runtime.

2. Aktifkan koleksi sinyal deteksi, bukti, kesetaraan, dan kesehatan sebelum mengirim injeksi; pastikan mekanisme pengelolaan sesuai mode telah aktif. Untuk mode imperatif, jeda *auto-sync*/*self-heal* pada `apps` lalu `invenio-bootstrap`, simpan spesifikasi asli, dan verifikasi penjedaan. Untuk mode GitOps, verifikasi keduanya aktif. Jalankan hanya satu skenario pada satu mode. Catat $t_0$, respons API pada $t_a$, dan verifikasi penerimaan injeksi tanpa menambahkan label atau identitas skenario ke sumber daya yang diamati.

3. Pertahankan koleksi berkala satu detik tanpa menyisipkan waktu tunggu agar manusia bereaksi atau agar drift bertahan selama durasi tertentu. Jika perubahan diterima tetapi sinyal singkat tidak teramati, laporkan keterbatasan observasi tanpa menganggap deteksi terjadi pada waktu nol.

4. Lanjutkan sampai pemulihan terkonfirmasi atau batas waktu pengamatan yang telah dibakukan tercapai. Simpan semua kejadian, jeda polling, kesalahan API, serta status penghentian.

5. Pulihkan kondisi dasar, verifikasi seluruh bidang/inventaris, kesehatan, HPA, tidak adanya objek karantina, adopsi runtime, serta keadaan aplikasi/data/queue. Untuk mode imperatif, pulihkan `invenio-bootstrap` lalu `apps` ke *automated sync*/*self-heal* asli dan verifikasi; kegagalan pemulihan dilaporkan sebagai `reset_failure`. Baru mulai pengulangan berikutnya. Kegagalan tindakan koreksi diteruskan, bukan ditelan. Tidak ada izin purge, restore, atau modifikasi data dari protokol ini.

Batas waktu maksimum pengamatan harus ditentukan dan dibekukan setelah pilot serta tinjauan pembimbing, sebelum pengumpulan formal. Tidak ada nilai batas waktu yang diklaim telah diuji dalam draf ini. Jika sinyal deteksi, bukti, atau pemulihan belum terjadi saat batas itu tercapai, hasil masing-masing ditandai tersensor kanan pada batas tersebut; waktu hilang tidak diganti nol dan pengulangan tidak dikeluarkan hanya karena pemulihannya gagal. Kegagalan injeksi, kegagalan instrumentasi, gangguan infrastruktur, dan tidak terjadinya deteksi merupakan kategori berbeda. Setiap kategori, jumlah pengulangan terdampak, alasan, serta aturan pengulangan yang telah disetujui dilaporkan.

Run dengan respons injeksi sukses tetapi tanpa bukti bahwa perubahan bidang sasaran pernah diterima tidak menjadi hasil nol; run ditandai sebagai injeksi tidak

valid dan dicatat. Jika perubahan diterima tetapi mekanisme tidak mendeteksinya sampai batas pengamatan, itu merupakan hasil deteksi tersensor, bukan kegagalan instrumen. Jika kolektor atau sumber data gagal sehingga status tidak dapat ditentukan, run ditandai kegagalan instrumentasi dan tidak ditafsirkan sebagai deteksi terlewat. Skenario F tetap diklasifikasikan sebagai pengamatan batas visibilitas dan tidak dinilai dengan asumsi pemulihan otomatis.

### 3.8 Pengulangan dan Analisis Data

Sampel formal adalah 15 pengulangan per skenario per mode: 10 pengulangan protokol R2 dan 5 pengulangan protokol R1, sehingga totalnya $7 \times 2 \times 15 = 210$ run pada satu kasus. Pembacaan R1 tetap disimpan pada setiap run sebagai bukti status, sedangkan 5 pengulangan R1 diarahkan pada protokol konsistensi. F dan G tetap menjalani protokol yang sama, tetapi hasil waktunya dilaporkan deskriptif dan tidak dipool ke estimasi efikasi. Sampling berlangsung pada interval nominal satu detik. Angka tersebut merupakan target yang direncanakan, bukan hasil analisis daya dan bukan jaminan kecukupan daya untuk setiap skenario (Dybå et al., 2006; Kitchenham and Madeyski, 2024). S1/S2 tidak dihitung dalam target ini. A sebagai kontrol HPA, F sebagai batas visibilitas, dan G sebagai kasus deskriptif tidak dimasukkan ke efikasi drift terkelola B–E.

Hasil disajikan per skenario dan per mode: jumlah run direncanakan, jumlah injeksi valid, jumlah kejadian teramati dan tersensor, ringkasan distribusi waktu, proporsi pemulihan pada batas pengamatan, R1, serta kesehatan terpisah. Nilai mentah yang hilang tidak diimputasi sebagai nol. Observasi yang tersensor tidak diam-diam dibuang agar hanya run yang berhasil pulih yang dibandingkan. Jika penyensoran terjadi, ringkasan dan uji yang tidak memperhitungkan penyensoran tidak ditafsirkan sebagai perbandingan lengkap; analisis komparatif untuk hasil tersebut harus ditetapkan sebelum melihat hasil, atau dilaporkan secara deskriptif tanpa klaim inferensial.

Karena mode berurutan pada stack yang sama, observasi tidak diperlakukan independen secara naif. Untuk hasil waktu yang teramati lengkap dan dapat dipasangkan berdasarkan blok skenario-pengulangan, uji nonparametrik berpasangan (misalnya Wilcoxon *signed-rank*) dapat dipertimbangkan; bila pemasangan tidak dapat dipertahankan, *Mann–Whitney U* dua sisi dilaporkan sebagai eksploratif. Ukuran efek (Cliff's $\delta$ atau Hodges–Lehmann) dilaporkan bersama interval kepercayaan bootstrap. Waktu deteksi dan pemulihan yang tersensor kanan dianalisis dengan Kaplan–

Meier serta median/*restricted mean survival*; nilai tersensor tidak diimputasi nol. Jika nilai $p$ digunakan, ambang nominalnya adalah $\alpha = 0{,}05$ dengan penyesuaian Holm untuk keluarga perbandingan waktu yang dibakukan sebelum pengumpulan formal. Definisi keluarga uji dan penanganan penyensoran harus ditinjau bersama pembimbing sebelum protokol dibekukan; tidak ada klaim konfirmatoris dari draf ini. Penggabungan lintas skenario tidak menjadi analisis utama karena bentuk dan dampak intervensinya berbeda. R1, kesehatan, dan kasus F/G dilaporkan deskriptif; tidak ada hasil signifikansi atau besar efek yang dinyatakan sebelum data dikumpulkan.

## 3.9 Ancaman terhadap Validitas dan Keterbatasan Rancangan

### 3.9.1 Validitas Konstruk

Validitas konstruk dibatasi oleh tujuh skenario yang dipilih, definisi bidang yang dibandingkan, interval polling, batas waktu, dan kriteria kesehatan. R1 mengukur kesetaraan bidang terpilih terhadap acuan, bukan kebenaran semantik atau keamanan konfigurasi. Pengamatan polling memberikan waktu deteksi pada resolusi instrumen, bukan waktu kejadian kontinu yang persis. R3 tidak diukur karena audit API server tidak tersedia, sehingga klaim keterlacakan tidak dapat dibuat. Skenario A, F, dan G tidak dapat digeneralisasi sebagai ukuran *self-healing* untuk sumber daya terkelola.

### 3.9.2 Validitas Internal

Validitas internal dapat terancam oleh perbedaan beban klaster dari waktu ke waktu, perubahan konfigurasi selain intervensi, ketidaksetaraan inventaris, serta perbedaan jadwal rekonsiliasi. Karena kedua mode berurutan pada satu stack, tidak ada isolasi penuh: beban latar dan keadaan klaster dapat berubah antarblok. Urutan pengujian yang diacak, nilai dasar yang dibekukan, pencatatan keadaan, penjedaan/pemulihan ArgoCD, dan gerbang kesiapan dirancang untuk mengurangi ancaman tersebut, tetapi tidak menjamin bahwa satu-satunya perbedaan adalah paradigma abstrak. Perbandingan tetap mengikutsertakan perbedaan implementasi yang melekat pada pemeriksa/korektor `kubectl` dan ArgoCD.

### 3.9.3 Validitas Eksternal

Validitas eksternal dibatasi pada satu klaster, satu stack InvenioRDM produksi-like, satu beban uji formal multi-service InvenioRDM, satu alat GitOps, serta konfigurasi dan versi yang benar-benar digunakan. Studi ini adalah studi kasus tunggal; hasilnya tidak dapat digeneralisasi ke semua aplikasi, klaster, atau alat GitOps. Pilot S1/S2 historis/offline tidak memperluas cakupan inferensi.

### 3.9.4 Validitas Kesimpulan

Jumlah pengulangan per sel terbatas (15 run; 10 untuk R2 dan 5 untuk R1) dan tidak dijamin memiliki daya statistik per skenario. Uji nonparametrik, ukuran efek, interval kepercayaan, dan pelaporan penyensoran dipakai agar kesimpulan tidak bergantung pada asumsi normalitas atau independensi yang tidak terpenuhi. Kegagalan, data tersensor, dan ketidakpastian dilaporkan; tidak ada klaim ekivalensi atau superioritas universal. Reliabilitas bergantung pada konsistensi kolektor, sinkronisasi jam, cakupan log, dan kemampuan mengulang reset. Sampai gerbang tersebut dipenuhi, hasil eksperimen belum tersedia dan protokol tetap berstatus draf.

# BAB IV
# HASIL DAN ANALISIS

## 4.1 Status Data dan Provenans

Pengumpulan data formal telah diselesaikan pada satu stack *workload* multi-service InvenioRDM produksi-like di namespace `invenio`. Dari 210 sel yang direncanakan, seluruh 210 sel mencapai status terminal tanpa sel yang tertunda. Berkas mentah memuat 211 baris: 210 sel rencana ditambah satu upaya ulang kegagalan *baseline* pra-injeksi yang diizinkan protokol. Analisis menerima 207 baris dan mengecualikan empat baris.

Empat pengecualian tersebut bersifat instrumentasi dan lingkungan, bukan hasil pengukuran: tiga baris berstatus `status_not_analyzable` akibat kegagalan instrumentasi (kolektor tidak menghasilkan sampel awal, atau pembacaan metrik node gagal karena layanan metrics-server tidak tersedia), dan satu baris berstatus `batch_stop` karena sel terhenti di tengah proses. Tidak ada nilai yang diimputasi, tidak ada observasi tersensor yang diubah menjadi kegagalan, dan tidak ada nilai yang hilang yang diganti dengan nol.

Identitas apparatus yang dibekukan sebelum pengumpulan dan dipakai tanpa perubahan adalah sebagai berikut.

Tabel 4.1: Identitas apparatus yang dibekukan

| Komponen | Nilai |
|---|---|
| Revisi sumber GitOps | `81ddd259780e0e6aef333811a0126f86d70a094e` |
| SHA-256 hasil render (27 objek) | `a5efd711d661438590a730291c41b70d1c668b2672d51a54f68c0ff6` |
| Revisi instrumen | `3572f3c1c11d3abc8b63882c4348b1b803d4256f` |
| *Image digest* aplikasi | `ghcr.io/vityasyyy/invenio-ugm@sha256:52645b7b...f343a357` |
| *Seed* | `20261029` |
| Urutan mode | gitops, lalu imperatif (berurutan, tidak paralel) |
| Urutan skenario | E, A, G, D, C, F, B |
| Konvensi waktu | `completed-observation-v1` |

Analisis dijalankan terhadap berkas yang dibekukan tersebut dengan *seed* bootstrap yang sama, sehingga hasil bersifat dapat direproduksi dari identitas pada Ta-

bel 4.1.

## 4.2 Format Pelaporan E1: Kesetaraan dan Kesehatan (R1)

R1 membandingkan skor kesetaraan bidang terpilih pada empat sumber daya yang dideklarasikan, pada skala 0–100, untuk lima pengulangan protokol R1 (pengulangan 11–15) per skenario per mode. Tabel 4.2 menyajikan median skor pada tiga titik: keadaan dasar sebelum injeksi, pengamatan pertama setelah injeksi, dan keadaan akhir setelah pemulihan.

Tabel 4.2: Median skor kesetaraan bidang terpilih (R1), format dasar / pascainjeksi / pascapemulihan

| Skenario | GitOps ($n = 5$) | Imperatif ($n = 5$) |
|---|---|---|
| A | 100 / 100 / 100 | 100 / 100 / 100 |
| B | 100 / 75 / 100 | 100 / 75 / 100 |
| C | 100 / 75 / 100 | 100 / 75 / 100 |
| D | 100 / 75 / 100 | 100 / 75 / 100 |
| E | 100 / 75 / 100 | 100 / 75 / 100 |
| F | 100 / 75 / 100 | 100 / 75 / 100 |
| G | 100 / 75 / **75** | 100 / 75 / 100 |

Pada skenario A hingga F, drift terdeteksi pada kedua mode—skor turun menjadi 75—dan kembali ke 100 setelah pemulihan. Skenario A tetap 100 sepanjang pengamatan, konsisten dengan keputusan mengecualikan `/spec/replicas` dari deklarasi karena bidang tersebut dimiliki HPA, bukan oleh mekanisme rekonsiliasi konfigurasi.

Skenario G merupakan satu-satunya asimetri: mode GitOps tidak mengembalikan anotasi pada SealedSecret ke keadaan yang dideklarasikan, sedangkan pemeriksa imperatif melakukannya. Temuan ini bukan kegagalan instrumentasi, melainkan perbedaan cakupan rekonsiliasi yang dapat dijelaskan: anotasi tersebut tidak termasuk himpunan bidang yang dikelola ArgoCD pada objek itu. Karena itu G dilaporkan sebagai kasus deskriptif dan tidak digabungkan ke dalam populasi efikasi.

## 4.3 Format Pelaporan E2: Waktu (R2)

R2 membandingkan tiga luaran waktu pada pengulangan 1–10: durasi deteksi ($t_d - t_0$), ketersediaan bukti yang dapat diverifikasi ($t_e - t_0$), dan pemulihan konfigurasi penuh ($t_r - t_0$). Seluruh waktu dihitung dari batas pembacaan yang tercatat kolektor dan tidak diinterpolasi melewati jeda pengambilan sampel. Observasi yang tidak mencapai titik akhir dalam jendela pengamatan dilaporkan sebagai tersensor kanan dan tidak diimputasi. Sebaran status pelaporan per blok dan luaran dirangkum pada Tabel 4.3.

Tabel 4.3: Jumlah observasi per luaran waktu dan status pelaporan

| Blok | Luaran | Teramati | Tersensor kanan | Tidak teramati sebelum berhenti |
|---|---|---|---|---|
| R2 | deteksi | 121 | 10 | 6 |
| R2 | bukti | 134 | 0 | 3 |
| R2 | pemulihan | 127 | 10 | 0 |
| R1 | deteksi | 61 | 5 | 4 |
| R1 | bukti | 70 | 0 | 0 |
| R1 | pemulihan | 65 | 5 | 0 |

Seluruh 15 kejadian sensoring pada luaran pemulihan berasal dari mode GitOps pada skenario G. Tidak ada nilai tersensor yang diimputasi. Titik akhir yang hilang karena pemulihan terkonfirmasi lebih awal dilaporkan terpisah sebagai *tidak teramati sebelum berhenti*; nilai tersebut bukan observasi tersensor dan tidak disertakan dalam uji numerik maupun analisis kelangsungan hidup.

### 4.3.1 Efikasi pemulihan drift terkelola (B–E)

Populasi efikasi terdiri atas skenario B, C, D, dan E, masing-masing dengan tiga luaran waktu, sehingga membentuk dua belas hipotesis yang telah ditetapkan sebelum pengumpulan data. Seluruh pengujian bersifat dua sisi dan dikoreksi Holm pada $\alpha = 0{,}05$. Tabel 4.4 menyajikan hasilnya.

Tabel 4.4: Perbandingan efikasi B–E: nilai $p$ terkoreksi Holm dan ukuran efek

| Sk. | Luaran | $n$ | $p$ (Holm) | $\delta$ | IK 95% $\delta$ | $\Delta$ median (detik) |
|---|---|---|---|---|---|---|
| B | deteksi | 10 | 0,0176 | 1,00 | [1,00; 1,00] | 4,27 |
| B | bukti | 10 | 0,0176 | 1,00 | [1,00; 1,00] | 4,05 |
| B | pemulihan | 10 | 0,0195 | 0,48 | [−0,08; 1,00] | 2,95 |
| C | deteksi | 8 vs 10 | 0,0054 | 1,00 | [1,00; 1,00] | tidak dilaporkan |
| C | bukti | 8 vs 10 | 0,0054 | 1,00 | [1,00; 1,00] | tidak dilaporkan |
| C | pemulihan | 8 vs 10 | 0,0176 | 0,75 | [0,25; 1,00] | tidak dilaporkan |
| D | deteksi | 4 vs 10 | 0,0176 | 1,00 | [1,00; 1,00] | tidak dilaporkan |
| D | bukti | 8 vs 10 | 0,0054 | 1,00 | [1,00; 1,00] | tidak dilaporkan |
| D | pemulihan | 10 | 0,0176 | 1,00 | [1,00; 1,00] | 5,05 |
| E | deteksi | 10 | 0,0176 | 1,00 | [1,00; 1,00] | 4,29 |
| E | bukti | 10 | 0,0176 | 1,00 | [1,00; 1,00] | 4,04 |
| E | pemulihan | 10 | 0,0176 | 0,86 | [0,64; 1,00] | 3,03 |

Seluruh dua belas perbandingan menolak hipotesis nol setelah koreksi Holm. Nilai $\Delta$ median berpasangan yang positif menunjukkan arah mode imperatif lebih lambat. Perlu dicatat bahwa skenario C dan D menggunakan uji Mann–Whitney U, bukan Wilcoxon *signed-rank* berpasangan, karena pasangan lengkap tidak mencukupi (masing-masing 8 dan 4 observasi GitOps). Peralihan tersebut mengikuti aturan cadangan yang telah ditetapkan sebelum analisis dan dilaporkan terbuka, bukan dipilih setelah melihat hasil.

Pola yang paling menonjol adalah perbedaan antara deteksi dan pemulihan. Selisih deteksi dan bukti mengelompok pada kisaran 4,0–4,3 detik, yang bersesuaian dengan interval *polling* pemeriksa imperatif. Karena itu selisih tersebut terutama mencerminkan perbedaan mekanisme pengamatan, bukan semata-mata ketanggapan pengendali. Sebaliknya, ukuran efek pemulihan lebih kecil daripada deteksi dan bukti pada skenario B ($\delta = 0{,}48$) dan C ($\delta = 0{,}75$). Dengan kata lain, mendeteksi drift jauh lebih mudah dipisahkan antar mode daripada benar-benar mengakhirinya.

## 4.4 R3: Tidak Diukur

R3 (bukti keterlacakan) tidak diukur dalam penelitian ini karena audit API server Kubernetes tidak tersedia. Tanpa audit, pelaku mutasi *out-of-band* tidak dapat direkonstruksi untuk kedua mode. R3 tidak dilaporkan sebagai nol, tidak dilaporkan sebagai tidak ada perbedaan, dan tidak diuji secara statistik. Keterlacakan dibahas sebagai keterbatasan pada Bab V dan sebagai pekerjaan lanjutan setelah audit diaktifkan.

## 4.5 Analisis yang Diterapkan

Analisis dilakukan per skenario dan tidak menggabungkan seluruh intervensi menjadi satu angka tunggal. Luaran waktu lengkap dianalisis dengan Wilcoxon *signed-rank* dua sisi menggunakan *exact sign-flip* untuk blok berpasangan berukuran paling banyak enam belas; selisih nol dieliminasi. Bila pasangan lengkap tidak tersedia, Mann–Whitney U dua sisi dengan koreksi ikatan dilaporkan sebagai analisis eksploratif. Ukuran efek dilaporkan sebagai Cliff's $\delta$ disertai interval kepercayaan bootstrap persentil, bersama selisih median berpasangan dan *rank-biserial*. Keluarga uji Holm terdiri atas dua belas hipotesis B–E $\times$ {deteksi, bukti, pemulihan}; hipotesis yang tidak dapat diuji diperlakukan konservatif sebagai $p = 1$. Untuk luaran yang tersensor, Kaplan–Meier dan *restricted mean survival time* digunakan dengan horizon sebesar batas sensoring terkecil pada kelompok yang bersangkutan.

Rancangan ini merupakan studi kasus tunggal dengan mode berurutan pada stack yang sama. Karena itu tidak ada klaim konfirmatori, dan seluruh nilai $p$ maupun ukuran efek ditafsirkan sebagai bukti eksploratif pada satu kasus, bukan sebagai estimasi populasi.

## 4.6 Batas Interpretasi

RQ1 hanya dapat dijawab terhadap inventaris dan bidang yang ditetapkan; kesetaraan skor bukan bukti bahwa konfigurasi benar secara semantik. RQ2 membedakan tiga hal yang tidak boleh dilebur: sinyal deteksi, tersedianya bukti yang dapat diverifikasi, dan kembalinya konfigurasi ke keadaan yang diinginkan. Kesehatan layanan tetap merupakan dimensi terpisah dan tidak disimpulkan dari skor kesetaraan.

Skenario A mempertahankan HPA pada kedua mode; respons replika bukan keberhasilan ArgoCD memulihkan bidang yang diabaikan. Skenario F dalam penelitian ini didefinisikan sebagai drift `targetPort` pada Service yang dideklarasikan, sehingga kedua mekanisme dapat memperbaikinya; F dilaporkan terpisah dan tidak dimasukkan ke keluarga efikasi karena dua belas hipotesis telah ditetapkan sebelum pengumpulan data, sehingga menambahkannya setelah melihat hasil akan memperluas keluarga secara *post hoc*. Skenario G merupakan kasus deskriptif non-ArgoCD dan dilaporkan tersendiri. Ketidakhadiran status OutOfSync bukan berarti deteksi bernilai nol.

Kedua mode menggunakan satu stack produksi-like secara berurutan, sehing-

ga perlakuan tidak terisolasi dari beban latar, keadaan data bersama, dan dependensi yang sama. Efek urutan dimitigasi melalui urutan berbasis *seed*, gerbang kesiapan *baseline*, prosedur reset, dan jeda antar-run, tetapi tidak dihilangkan. Selain itu, kedua mode berbagi satu domain kegagalan penyimpanan dan tidak ada salinan off-site. Empat pengecualian instrumentasi juga mengurangi jumlah observasi pada sel terkait, dan hal ini dilaporkan apa adanya alih-alih ditambal.

Dengan batasan tersebut, hasil pada bab ini menunjukkan bahwa pada apparatus dan protokol yang dibekukan, kedua mode mendeteksi dan memulihkan drift konfigurasi pada seluruh skenario terkelola, dengan perbedaan waktu deteksi yang konsisten dengan perbedaan mekanisme pengamatan, dan dengan perbedaan waktu pemulihan yang jauh lebih kecil. Hasil ini tidak dapat digeneralisasi menjadi klaim keunggulan salah satu mode secara umum, dan tidak membuktikan kesetaraan semantik konfigurasi, keterlacakan mutasi, maupun pemulihan tingkat aplikasi.

# BAB V
# KESIMPULAN DAN SARAN

## 5.1 Kesimpulan

Penelitian ini membandingkan mode imperatif terotomasi berbasis `kubectl` dengan mode GitOps berbasis ArgoCD pada satu stack *workload* multi-service InvenioRDM produksi-like. Kedua mode dijalankan berurutan pada stack yang sama terhadap apparatus yang dibekukan, dengan tujuh skenario drift, lima belas pengulangan per skenario per mode, dan total 210 sel. Dari 210 sel tersebut, 207 baris dianalisis dan empat baris dikecualikan sebagai kegagalan instrumentasi serta lingkungan yang dilaporkan secara terbuka.

### 5.1.1 Jawaban RQ1: kesetaraan konfigurasi

Pada skenario A hingga F, kedua mode mengembalikan skor kesetaraan bidang terpilih dari 100 pada keadaan dasar, turun ke 75 saat drift terdeteksi, lalu kembali ke 100 setelah pemulihan. Dengan demikian, untuk himpunan bidang yang dideklarasikan, kedua mode mencapai keadaan akhir yang setara pada seluruh skenario terkelola.

Satu pengecualian bersifat sistematis dan dapat dijelaskan. Pada skenario G, mode GitOps tidak mengembalikan anotasi pada SealedSecret ke keadaan yang dideklarasikan, sedangkan mode imperatif melakukannya. Perbedaan ini bukan kegagalan acak, melainkan konsekuensi cakupan rekonsiliasi: anotasi tersebut tidak termasuk himpunan bidang yang dikelola ArgoCD pada objek itu, sementara pemeriksa imperatif membandingkan bidang yang dideklarasikan secara umum. Temuan ini menunjukkan bahwa kesetaraan akhir bergantung pada apa yang didefinisikan sebagai keadaan yang diinginkan, bukan semata pada mode yang digunakan.

Perlu ditegaskan bahwa kesetaraan skor tidak membuktikan kebenaran semantik konfigurasi. Kedua mode dapat mencapai keadaan yang identik dan sama-sama salah apabila deklarasi sumbernya keliru. Penelitian ini mengukur kesesuaian terhadap deklarasi, bukan kualitas deklarasi itu sendiri.

### 5.1.2 Jawaban RQ2: waktu deteksi, bukti, dan pemulihan

Seluruh dua belas perbandingan yang telah ditetapkan sebelum pengumpulan data menolak hipotesis nol setelah koreksi Holm pada $\alpha = 0{,}05$. Namun arah dan besarnya perbedaan tidak seragam, dan perbedaan itu perlu dibaca secara hati-hati.

Pada luaran deteksi dan ketersediaan bukti, ukuran efek mencapai nilai maksimum yang mungkin ($\delta = 1{,}00$), dan selisih median berpasangan mengelompok pada kisaran 4,0–4,3 detik. Angka tersebut bersesuaian dengan interval *polling* pemeriksa imperatif yang digunakan dalam penelitian ini. Karena itu, selisih deteksi terutama mencerminkan perbedaan mekanisme pengamatan—pengambilan sampel berkelanjutan pada GitOps dibandingkan pemeriksaan berkala pada mode imperatif—dan bukan semata-mata ketanggapan pengendali dalam mengenali penyimpangan.

Pada luaran pemulihan, ukuran efek lebih kecil: $\delta = 0{,}48$ untuk skenario B dan $\delta = 0{,}75$ untuk skenario C, meskipun tetap tinggi pada D dan E. Pola ini merupakan temuan utama penelitian. Mendeteksi drift jauh lebih mudah dipisahkan antar mode daripada benar-benar mengakhirinya. Setelah penyimpangan diketahui, jalur pemulihan kedua mode konvergen menuju hasil yang serupa, sehingga keunggulan waktu yang besar pada tahap deteksi tidak otomatis berlanjut ke tahap pemulihan.

Dengan demikian, jawaban RQ2 bukan pernyataan bahwa salah satu mode lebih unggul secara umum, melainkan bahwa keunggulan waktu terkonsentrasi pada tahap awal siklus rekonsiliasi dan menyusut pada tahap akhir. Implikasi praktisnya: pemilihan mode sebaiknya mempertimbangkan berapa lama penyimpangan boleh bertahan, bukan hanya seberapa cepat penyimpangan terdeteksi.

### 5.1.3 R3 dan batas kesimpulan

R3 (bukti keterlacakan) tidak diukur karena audit API server Kubernetes tidak tersedia. Tanpa audit, pelaku mutasi *out-of-band* tidak dapat direkonstruksi untuk kedua mode. R3 tidak dilaporkan sebagai nol, tidak dilaporkan sebagai tidak ada perbedaan, dan tidak diuji secara statistik.

Secara keseluruhan, penelitian ini tidak menyimpulkan keunggulan salah satu mode secara umum, tidak menyimpulkan kelayakan adopsi, dan tidak menyimpulkan kesetaraan semantik konfigurasi. Yang disimpulkan adalah: pada apparatus, protokol, dan cakupan bidang yang diuji, kedua mode memulihkan drift terkelola menuju keadaan yang dideklarasikan, dengan perbedaan waktu deteksi yang konsisten dengan perbedaan mekanisme pengamatan, dan dengan perbedaan waktu pemulihan yang ja-

uh lebih kecil. Skenario A dilaporkan sebagai kontrol HPA, F sebagai drift Service terdeklarasi yang dilaporkan terpisah, dan G sebagai kasus deskriptif non-ArgoCD; ketiganya tidak dipool ke dalam estimasi efikasi B–E.

## 5.2 Saran

### 5.2.1 Bagi praktisi

Pertama, pilih mode berdasarkan lama toleransi penyimpangan, bukan berdasarkan kecepatan deteksi semata. Apabila organisasi memiliki jendela toleransi yang panjang dan volume perubahan yang rendah, keunggulan deteksi GitOps tidak banyak mengubah risiko operasional. Sebaliknya, apabila penyimpangan harus terdeteksi dalam hitungan detik, mekanisme pengamatan berkelanjutan memberi manfaat yang jelas.

Kedua, jangan mengasumsikan bahwa rekonsiliasi otomatis mencakup seluruh bidang objek. Temuan skenario G menunjukkan bahwa bidang di luar himpunan yang dikelola—seperti anotasi tertentu—dapat bertahan meskipun objek secara umum telah selaras. Organisasi perlu menetapkan secara eksplisit bidang mana yang menjadi tanggung jawab otomasi dan bidang mana yang tidak, lalu memverifikasi asumsi tersebut, bukan mengandalkannya.

Ketiga, siapkan jalur pemulihan yang diuji terpisah dari jalur deteksi. Karena pemulihan tidak secara otomatis secepat deteksi, alarm yang cepat tanpa prosedur remediasi yang teruji hanya memindahkan penundaan ke tahap berikutnya.

Keempat, ukur kesiapan apparatus sebelum menjalankan pengukuran apa pun. Pengalaman dalam penelitian ini menunjukkan bahwa kegagalan yang paling banyak menghabiskan waktu bukan berasal dari perbedaan mode, melainkan dari ketersediaan API, kesegaran metrik, dan stabilitas koneksi saat pengukuran berlangsung.

### 5.2.2 Bagi peneliti

Protokol perlu dibakukan sebelum pengumpulan data, termasuk definisi keadaan yang diinginkan, daftar bidang yang dibandingkan, aturan penyensoran, keluarga uji, dan kriteria penerimaan injeksi. Analisis pada penelitian ini menggunakan aturan yang telah ditetapkan sebelum data dikumpulkan, termasuk aturan cadangan ketika pasangan lengkap tidak tersedia; pendekatan tersebut mencegah pemilihan uji secara *post hoc*.

Semua kategori kegagalan dan penyensoran perlu dipertahankan dan dilaporkan, bukan dibersihkan untuk memperoleh distribusi yang lebih menguntungkan. Empat pengecualian pada penelitian ini dilaporkan dengan identitas run masing-masing agar dapat diaudit.

Klaim komparatif perlu dibatasi pada prosedur, *workload*, klaster, dan resolusi instrumen yang benar-benar diuji. Hasil studi kasus tunggal tidak dapat digeneralisasi tanpa replikasi.

## 5.3 Keterbatasan Penelitian

Keterbatasan yang melekat pada rancangan ini meliputi penggunaan satu klaster, satu stack InvenioRDM produksi-like, dan satu implementasi GitOps. Kedua mode dijalankan berurutan pada stack yang sama, sehingga perlakuan tidak terisolasi dari beban latar, dependensi bersama, dan keadaan klaster yang berubah. Efek urutan dimitigasi melalui urutan berbasis *seed*, gerbang kesiapan *baseline*, prosedur reset, dan jeda antar-run, tetapi tidak dihilangkan.

Kedua mode berbagi satu domain kegagalan penyimpanan dan tidak memiliki salinan off-site. Mode imperatif yang diuji adalah satu otomasi `kubectl` tertentu, sehingga tidak mewakili semua cara kerja imperatif. Interval pengambilan sampel satu detik membatasi ketelitian waktu kejadian, dan ketidakpastian jam antar-host ditangani secara konservatif, bukan disertifikasi secara numerik. Target 210 run bukan jaminan daya statistik per skenario (Dybå et al., 2006; Kitchenham and Madeyski, 2024).

Skenario A adalah kontrol HPA, F adalah drift Service terdeklarasi yang dilaporkan terpisah karena keluarga dua belas hipotesis telah ditetapkan sebelum pengumpulan, dan G adalah kasus deskriptif yang bukan *self-heal* ArgoCD. Ketiganya tidak dipool ke estimasi efikasi B–E.

R3 tidak diukur karena audit API server Kubernetes tidak tersedia. Pengukuran kesetaraan juga tidak menilai apakah konfigurasi yang diinginkan aman atau benar secara semantik. Empat pengecualian instrumentasi mengurangi jumlah observasi pada sel terkait. Selain itu, protokol belum memperoleh tinjauan pembimbing, sehingga posisi penelitian ini adalah studi kasus eksploratif, bukan studi konfirmatori.

## 5.4 Pekerjaan Lanjutan

### 5.4.1 Replikasi dan validitas eksternal

Prioritas pertama adalah replikasi pada beberapa klaster dan beberapa stack independen. Studi ini menggunakan satu stack berurutan; replikasi pada setidaknya tiga stack paralel yang terisolasi memungkinkan pemisahan efek urutan dari efek mode, serta memungkinkan uji berpasangan yang sebenarnya alih-alih blok berurutan. Replikasi juga memungkinkan estimasi varians antar-stack, yang tidak dapat diperoleh dari satu kasus.

Prioritas berikutnya adalah membandingkan lebih dari satu implementasi GitOps, misalnya Flux atau ArgoCD dengan konfigurasi manajemen bidang yang berbeda, menggunakan definisi metrik yang sama. Temuan skenario G menunjukkan bahwa perilaku rekonsiliasi bergantung pada himpunan bidang yang dikelola; membandingkan beberapa implementasi akan menguji seberapa umum pola tersebut.

### 5.4.2 Pengukuran yang dapat diperluas dengan sumber daya tambahan

Apabila tersedia sumber daya komputasi dan anggaran yang lebih besar, beberapa pengukuran yang saat ini berada di luar jangkauan dapat dilakukan.

1. **R3 melalui audit API server.** Mengaktifkan audit Kubernetes dengan kebijakan yang mencatat mutasi, lalu merekonstruksi pelaku perubahan untuk kedua mode. Ini memungkinkan pertanyaan keterlacakan dijawab secara empiris, bukan ditinggalkan sebagai batasan.

2. **Resolusi waktu yang lebih tinggi.** Mengurangi interval pengambilan sampel menjadi di bawah satu detik dan menambahkan sinkronisasi jam presisi (misalnya PTP) akan memisahkan komponen cadangan mekanisme dari komponen ketanggapan pengendali secara lebih tegas. Pada penelitian ini, selisih deteksi sekitar empat detik berimpit dengan interval *polling*, sehingga kedua komponen tersebut belum dapat dipisahkan sepenuhnya.

3. **Beban pengguna yang realistis.** Menjalankan percobaan pada trafik pengguna nyata atau beban sintetis yang terkalibrasi memungkinkan pengukuran dampak pemulihan terhadap layanan, bukan hanya terhadap konfigurasi. Ini juga menguji apakah pemulihan yang cepat tetap cepat ketika klaster berada di bawah tekanan.

4. **Volume data dan skala klaster.** Mengulang pada volume objek dan jumlah node yang jauh lebih besar akan menguji apakah perbedaan mode berskala atau justru menyempit ketika biaya rekonsiliasi meningkat.

5. **Dayaguna statistik per skenario.** Menambah jumlah pengulangan per skenario berdasarkan perhitungan daya yang menggunakan varians yang teramati pada studi ini. Studi ini memberikan estimasi varians yang dapat dipakai untuk perencanaan tersebut.

6. **Skenario drift yang lebih kompleks.** Menambahkan drift berganda yang saling bergantung, drift yang melibatkan beberapa objek sekaligus, dan drift yang bersaing dengan rekonsiliasi lain, untuk menguji perilaku ketika penyimpangan tidak berdiri sendiri.

### 5.4.3 Mengurangi batasan protokol

Beberapa batasan pada penelitian ini berasal dari keputusan desain yang diambil untuk menjaga kelayakan, bukan dari keterbatasan yang tak terhindarkan. Dengan kelonggaran yang lebih besar, batasan berikut dapat dikurangi.

1. **Stack kembar yang terisolasi.** Dua stack identik yang berjalan paralel akan menghilangkan efek urutan dan memisahkan perlakuan dari beban latar bersama. Konsekuensinya adalah kebutuhan kapasitas dua kali lipat dan hilangnya sebagaian validitas ekologis, karena kedua stack tidak lagi berbagi keadaan yang sama; pertukaran ini perlu dinyatakan secara eksplisit, bukan dianggap perbaikan murni.

2. **Jendela pengamatan yang lebih panjang.** Memperpanjang batas pengamatan akan mengurangi jumlah kejadian tersensor dan mempersempit selang kepercayaan pada luaran pemulihan. Pada penelitian ini seluruh sensoring pemulihan berasal dari satu skenario, sehingga perpanjangan jendela terutama bermanfaat untuk memahami kasus tersebut.

3. **Infrastruktur independen dan salinan off-site.** Memisahkan penyimpanan cadangan dari domain kegagalan utama akan memungkinkan pengujian pemulihan yang sesungguhnya, termasuk skenario kehilangan penyimpanan primer. Penelitian ini tidak dapat menguji hal tersebut karena keduanya berbagi satu server NFS.

4. **Jam yang tersertifikasi.** Menggunakan sumber waktu yang tersertifikasi memungkinkan klaim selang waktu absolut, bukan hanya perbandingan relatif dengan penanganan konservatif.

5. **Lingkungan yang tenang dan terkendali.** Menjalankan percobaan pada klaster khusus tanpa beban lain akan meningkatkan presisi, dengan mengorbankan kesetiaan terhadap kondisi operasional nyata.

### 5.4.4 Agenda yang diprioritaskan

Tabel 5.1 merangkum prioritas pekerjaan lanjutan beserta alasan dan perkiraan sumber daya yang dibutuhkan. Urutan ini disusun berdasarkan rasio antara nilai inferensial dan biaya pelaksanaan.

Tabel 5.1: Prioritas pekerjaan lanjutan

| Prioritas | Pekerjaan | Nilai utama | Sumber daya |
|---|---|---|---|
| 1 | Mengaktifkan audit API server untuk R3 | Menjawab pertanyaan yang saat ini tidak terukur | Rendah |
| 2 | Replikasi pada tiga stack terisolasi | Memisahkan efek urutan dari efek mode | Tinggi |
| 3 | Sinkronisasi jam presisi dan sampling sub-detik | Memisahkan cadangan mekanisme dari ketanggapan pengendali | Sedang |
| 4 | Menambah pengulangan berdasarkan daya teramati | Mempersempit selang kepercayaan pada luaran pemulihan | Sedang |
| 5 | Beban pengguna realistis | Menguji dampak terhadap layanan, bukan hanya konfigurasi | Sedang |
| 6 | Membandingkan beberapa implementasi GitOps | Menguji generalitas temuan skenario G | Sedang |
| 7 | Infrastruktur cadangan independen | Menguji pemulihan lintas domain kegagalan | Tinggi |
| 8 | Skenario drift berganda dan saling bergantung | Menguji perilaku di luar drift tunggal | Sedang |

Secara keseluruhan, arah pekerjaan lanjutan yang paling bernilai bukan menambah jumlah pengamatan pada apparatus yang sama, melainkan memisahkan va-

riabel yang pada penelitian ini masih berimpit: mekanisme pengamatan dari ketanggapan pengendali, efek urutan dari efek mode, serta cakupan bidang terkelola dari cakupan bidang yang diasumsikan. Pemisahan tersebut membutuhkan sumber daya dan kelonggaran yang lebih besar, tetapi akan mengubah temuan dari deskriptif menjadi penjelasan.

# DAFTAR PUSTAKA


R. Aghili, Q. Qin, H. Li, and F. Khomh. Understanding web application workloads and their applications: Systematic literature review and characterization. In *2024 IEEE International Conference on Web Services (ICWS)*. IEEE, 2024.

Argo Project. Argocd documentation, 2024. URL `https://argo-cd.readthedocs.io`.

Argo Project. Argo cd documentation: Automated sync policy. `https://argo-cd.readthedocs.io/en/stable/user-guide/auto_sync/`, 2025. Dokumentasi resmi perangkat lunak.

Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *ACM Queue*, 14(1):70–93, 2016.

Scott Chacon and Ben Straub. *Pro Git*. Apress, 2nd edition, 2014.

CNCF GitOps Working Group. Opengitops principles v1.0.0. `https://opengitops.dev`, 2021.

Thomas D. Cook and Donald T. Campbell. *Quasi-Experimentation: Design and Analysis Issues for Field Settings*. Houghton Mifflin, 1979.

M. D'Amore. Gitops and argocd: Continuous deployment and maintenance of a full stack application in a hybrid cloud kubernetes environment. Master's thesis, Politecnico di Torino, 2021.

Tore Dybå, Vigdis By Kampenes, and Dag I. K. Sjøberg. A systematic review of statistical power in software engineering experiments. *Information and Software Technology*, 48(8):745–755, 2006. doi: 10.1016/j.infsof.2005.08.009.

Nicole Forsgren, Jez Humble, and Gene Kim. *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.

Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.

Barbara Kitchenham and Lech Madeyski. Recommendations for analysing and meta-analysing small sample size software engineering experiments. *Empirical Software Engineering*, 29(6), 2024. doi: 10.1007/s10664-024-10504-1.

Thomas A. Limoncelli, Strata R. Chalup, and Christina J. Hogan. *The Practice of Cloud System Administration: Designing and Operating Large Distributed Systems*. Addison-Wesley Professional, 2nd edition, 2018.

Gerardo Lopez and Saloni Narang. Gitops in 2025: From old-school updates to the modern way. `https://www.cncf.io/blog/2025/06/09/gitops-in-2025-from-old-school-updates-to-the-modern-way/`, 2025. Artikel industri CNCF, bukan artikel peer-review; hanya untuk konteks.

M. E. Matubber. Managing 5g kubernetes infrastructures using gitops principles. Master's thesis, KTH Royal Institute of Technology, 2025.

Kief Morris. *Infrastructure as Code: Dynamic Systems for the Cloud Age*. O'Reilly Media, 2nd edition, 2020.

P. Kagganti Nataraja. A security-centric analysis of declarative and imperative deployment approaches in kubernetes-based application environments. Master's thesis, National College of Ireland, 2025.

E. Paavola. Managing multiple applications on kubernetes using gitops principles. Master's thesis, Turku University of Applied Sciences, 2021.

O. Pohjola. Mitigating configuration drift in infrastructure-as-code systems. Master's thesis, Aalto University, 2025.

A. Rahman, S. I. Shamim, D. B. Bose, et al. Security misconfigurations in open source kubernetes manifests: An empirical study. *ACM Transactions on Software Engineering and Methodology*, 2023.

Per Runeson and Martin Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164, 2009. doi: 10.1007/s10664-008-9102-8.

D. Schlette, P. Empl, M. Caselli, T. Schreck, and G. Pernul. Do you play it by the books? a study on incident response playbooks and influencing factors. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024.

Michael L. Scott. *Programming Language Pragmatics*. Morgan Kaufmann, 4th edition, 2015.

R. Shrestha and A. A. N. Ali. Configuration management in kubernetes environments: A gitops approach. In *2024 IEEE/ACM 17th International Conference on Utility and Cloud Computing (UCC)*. IEEE, 2024.

H. Siddiqui, F. Khendek, and M. Toeroe. Microservices based architectures for iot systems—state-of-the-art review. *Internet of Things*, 2023.

Ervina Utami and Hanif Al Fatta. Analysis on the use of declarative and pull-based deployment models on gitops using argo cd. In *2021 4th International Conference on Information and Communications Technology (ICOIACT)*. IEEE, 2021.

Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at google with borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15)*, pages 1–17. ACM, 2015.

Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2nd edition, 2012.

Billy Yuen, Alexander Matyushentsev, Jesse Suen, and Thomas Ekenstam. *GitOps and Kubernetes: Continuous Deployment with Argo CD, Jenkins X, and Flux*. O'Reilly Media, 2021. ISBN 978-1492094877.

Yue Zhang, Uchswas Paul, Marcelo d'Amorim, and Akond Rahman. Configuration defects in kubernetes. *IEEE Transactions on Software Engineering*, 52(3):1125–1144, 2026. doi: 10.1109/TSE.2026.3659785.

# LAMPIRAN A

# LAMPIRAN A

Lampiran ini memuat artefak, perintah, dan skema data yang benar-benar digunakan pada pengumpulan data formal, beserta pemetaan antara keterbatasan penelitian dan cara penanganannya. Seluruh nilai yang dicantumkan telah disanitasi: tidak ada nilai Secret, kredensial, nama objek privat, maupun URL bertanda tangan di dalam lampiran ini.

## 1.1 Identitas Apparatus yang Dibekukan

Tabel 1.1: Identitas apparatus pada pengumpulan formal

| Komponen | Nilai |
|---|---|
| Revisi sumber GitOps | `81ddd259780e0e6aef333811a0126f86d70a094e` |
| SHA-256 render | `a5efd711d661438590a730291c41b70d1c668b2672d51a54f68c0ff6` |
| Jumlah objek render | 27 |
| Revisi instrumen | `3572f3c1c11d3abc8b63882c4348b1b803d4256f` |
| *Image digest* aplikasi | `ghcr.io/vityasyyy/invenio-ugm@sha256:52645b7b...f343a357` |
| *Seed* | `20261029` |
| Jumlah sel | 210 (7 skenario $\times$ 2 mode $\times$ 15 pengulangan) |
| Konvensi waktu | `completed-observation-v1` |

## 1.2 Perintah Eksekusi

Pengumpulan dijalankan melalui satu perintah batch yang memakai ulang jalur harian pemeriksa, sehingga gerbang keamanan tetap berada di satu tempat. Kredensial dibaca dari berkas privat bermode 0600 di luar repositori dan disuntikkan ke lingkungan proses anak; nilainya tidak pernah dicetak, tidak ditulis ke berkas bukti, dan tidak muncul pada argumen perintah.

```
python3 -B experiments/live/run_matrix.py \
  --batch-id formal-20261029 \
  --arms imperative gitops \
  --scenarios A B C D E F G \
  --reps 15 --seed 20261029 \
```

```
  --cooldown-seconds 60 --baseline-timeout 600 --baseline-retries 1
  --namespace invenio --app invenio-bootstrap \
  --live --confirm-single-stack --production-like \
  --out experiments/formal-20261029 \
  --render-file <frozen-render.json> \
  --source-revision 81ddd259780e0e6aef333811a0126f86d70a094e \
  --render-sha256 a5efd711d661438590a730291c41b70d1c668b2672d51a54f
```

Analisis dijalankan dengan perintah berikut; keluaran berupa satu berkas JSON dan enam tabel LaTeX.

```
python3 -B experiments/analysis/analyze_live.py \
  --in experiments/formal-20261029/formal-20261029.analysis.jsonl \
  --rows experiments/formal-20261029/formal-20261029.rows.jsonl \
  --out experiments/analysis-formal-20261029 --seed 20261029
```

### 1.3 Skema Data per Run

Setiap run menulis satu berkas baris JSON dengan bidang berikut. Bidang waktu disimpan ganda: cap waktu UTC untuk keterbacaan dan waktu monotonik untuk perhitungan selang, sehingga perubahan jam dinding tidak memengaruhi durasi.

Tabel 1.2: Bidang utama pada berkas baris per run

| Bidang | Isi |
|---|---|
| `run_id`, `arm`, `scenario`, `rep` | Identitas run dan blok protokol (R2 untuk pengulangan 1–10, R1 untuk 11–15) |
| `status`, `stop_reason` | Status terminal run dan alasan berhenti |
| `detection_seconds`, `evidence_seconds`, `recovery_seconds` | Tiga luaran waktu; bernilai kosong bila tidak teramati, tidak pernah nol |
| `detection_status`, `evidence_status` | Teramati, tersensor kanan, atau tidak teramati sebelum berhenti |
| `r1_baseline`, `r1_first`, `r1_last` | Skor kesetaraan bidang terpilih pada tiga titik |
| `timing_convention` | Penanda konvensi waktu agar data antarkonvensi tidak ter- |

Setiap run juga menyimpan bukti pendukung berupa cuplikan keadaan dasar, kontrak deklarasi, catatan injeksi, cuplikan pengambilan sampel, kondisi pascasyarat, dan catatan pembersihan. Berkas-berkas ini yang memungkinkan audit ulang tanpa menjalankan eksperimen kembali.

## 1.4 Perlindungan Informasi

Snapshot dan bukti disanitasi sebelum disimpan. Nilai Secret tidak dimasukkan ke keluaran, tangkapan layar, log, maupun dataset. Pencocokan bidang dilakukan di memori; artefak yang dibagikan hanya memuat identitas bidang dan hasil kesetaraan, tanpa nilai rahasia maupun hash yang dapat dipakai untuk menebak isinya. Kredensial pengumpulan berada pada berkas privat di luar repositori dan tidak disertakan pada lampiran ini.

## 1.5 Pemetaan Keterbatasan dan Penanganannya

Tabel berikut menyatakan setiap keterbatasan, cara penanganan yang benar-benar dilakukan, dan sisa risiko yang tetap ada. Kolom terakhir penting: tidak ada keterbatasan yang diklaim hilang sepenuhnya.

Tabel 1.3: Keterbatasan, penanganan, dan sisa risiko

| Keterbatasan | Penanganan dalam penelitian | Sisa risiko |
|---|---|---|
| Satu klaster, satu stack, satu implementasi GitOps | Seluruh klaim dibatasi pada apparatus yang diuji; identitas apparatus dibekukan dan dilaporkan | Generalisasi ke lingkungan lain tidak dapat ditarik |
| Kedua mode berurutan pada stack yang sama | Urutan berbasis *seed*, gerbang kesiapan *baseline*, prosedur reset, dan jeda 60 detik; urutan dilaporkan | Efek urutan dan beban latar tidak dihilangkan, hanya dimitigasi |
| Tidak ada stack kembar terisolasi | Dinyatakan eksplisit sebagai batasan desain, bukan diklaim sebagai isolasi | Perlakuan tidak independen dari keadaan klaster bersama |
| Satu domain kegagalan penyimpanan | Dinyatakan bahwa penyimpanan primer dan cadangan berbagi server NFS yang sama | Pemulihan lintas kehilangan penyimpanan primer tidak diuji |
| Interval pengambilan sampel satu detik | Selisih deteksi dilaporkan bersama interpretasi bahwa nilainya bersesuaian dengan interval *polling* | Komponen cadangan mekanisme dan ketanggapan pengendali belum terpisah sepenuhnya |
| Ketidakpastian jam antar-host | Batas kesegaran 30 detik diterapkan secara konservatif; sampel dari masa depan dianggap anomali dan menghentikan run | Selang waktu absolut tidak disertifikasi secara numerik |
| Sensoring kanan | Dilaporkan sebagai tersensor, tidak diimputasi, dan dianalisis dengan Kaplan–Meier serta RMST | Jumlah kejadian tersensor pada satu skenario membatasi presisi luaran pemulihan |
| Empat baris dikecualikan | Setiap baris disebutkan dengan identitas run dan alasan; tidak ada penggantian nilai | Jumlah observasi pada sel terkait berkurang |
| Keluarga uji dua belas hipotesis | Ditetapkan sebelum pengumpulan; F tidak ditambahkan setelah melihat data | Pertanyaan tentang F tidak diuji secara inferensial |
| Uji berpasangan tidak selalu dapat dipakai | Aturan cadangan Mann–Whitney ditetapkan sebelum analisis dan dilaporkan terbuka | Daya statistik pada C dan D lebih rendah |
| R3 tidak diukur | Dilaporkan sebagai tidak diukur, bukan nol dan bukan tidak ada perbedaan | Pertanyaan keterlacakan tidak terjawab |
| Kesetaraan bukan kebenaran semantik | Dinyatakan bahwa skor mengukur kesesuaian terhadap deklarasi, bukan kualitas deklarasi | Konfigurasi yang salah dapat dicapai dengan rapi oleh kedua mode |
| Protokol belum ditinjau pembimbing | Dinyatakan sebagai studi kasus eksploratif, bukan konfirmatori | Posisi metodologis menunggu tinjauan |

## 1.6 Cara Membaca Hasil

Untuk pembaca yang tidak terbiasa dengan statistik, dua panduan berikut membantu menafsirkan Bab IV secara tepat.

Pertama, nilai $p$ yang kecil menunjukkan bahwa perbedaan yang teramati sulit dijelaskan oleh kebetulan, tetapi tidak menunjukkan besarnya perbedaan. Karena itu setiap perbandingan disertai ukuran efek. Pada penelitian ini seluruh perbanding-

an deteksi memiliki ukuran efek maksimum, sedangkan pemulihan tidak, dan justru perbedaan pada kolom ukuran efek itulah temuan yang paling penting.

Kedua, pengamatan tersensor bukan pengamatan gagal. Apabila pemulihan belum terjadi sampai batas pengamatan, yang diketahui adalah bahwa waktunya melebihi batas tersebut; nilainya tidak diketahui dan tidak diisi dengan angka karangan. Pada penelitian ini seluruh sensoring pemulihan berasal dari satu skenario pada satu mode, sehingga kejadian tersebut dilaporkan sebagai temuan, bukan dibuang.

## 1.7 Status Lampiran

Seluruh artefak yang dirujuk pada lampiran ini telah dihasilkan oleh eksperimen dan telah disanitasi. Dataset mentah, berkas bukti per run, dan tabel LaTeX hasil analisis disimpan di luar naskah ini bersama checksum masing-masing, sehingga integritasnya dapat diverifikasi ulang tanpa memuat data sensitif ke dalam dokumen yang dipublikasikan.